

Adaptive Hybrid RAG

Project Understanding + Interview Preparation Guide

Prepared for a B.Tech CSE (Core) fresher, VIT Bhopal
Based entirely on the uploaded project source code (adaptive-hybrid-rag-main.zip)

This document explains the project from zero, connects it to core Computer Science concepts, and prepares project-specific interview questions, cross-questions, and revision material. Every technical claim is grounded in the actual uploaded code. Where something could not be verified from the files, this is stated explicitly rather than guessed.

Project Name	Adaptive Hybrid RAG
Type	Research-oriented Retrieval-Augmented Generation (RAG) system
Core Idea	Learns a per-query BM25 / Dense retrieval weight instead of a fixed 50/50 blend
Backend	Python, FastAPI, SQLite, ChromaDB
Frontend	React 19, Vite, plain CSS, lucide-react
ML Component	ExtraTreesRegressor adaptive weight predictor (17 engineered features)
LLM	Qwen2.5:7B served locally through Ollama

TABLE OF CONTENTS

- Part 1 - Understanding the Project From Absolute Zero**
- Part 2 - Full Technology Inventory (What Is Actually Used)**
- Part 3 - File-by-File Explanation of Key Files**
- Part 4 - Every Technology Explained From Zero (What / Why / How / Where / Questions)**
- Part 5 - Core Computer Science Connections**
- Part 6 - Core Code Logic (Input -> Processing -> Output)**
- Part 7 - Database Design (SQLite + ChromaDB)**
- Part 8 - API / Backend Design**
- Part 9 - Frontend Design**
- Part 10 - Machine Learning / LLM / RAG Deep Dive**
- Part 11 - System Architecture**
- Part 12 - How To Run The Project**
- Part 13 - Realistic Project Challenges**
- Part 14 - Interview Questions By Category**
- Part 15 - Top 30 Most Important Interview Questions**
- Part 16 - Cross-Questions ("Why? How? What if?")**
- Part 17 - "What Did You Personally Do?" Preparation**
- Part 18 - Resume-Based Interview Questions**
- Part 19 - Honesty Check - Claims You Should NOT Make**
- Part 20 - Final Revision Sheet**

Understanding the Project From Absolute Zero

1.1 What is this project?

Adaptive Hybrid RAG is a document question-answering system. A user uploads a document (or the system already has a large pre-built knowledge base), asks a natural-language question, and the system finds the most relevant pieces of text and uses a local large language model (LLM) to write a grounded answer based only on that retrieved text. This pattern - find relevant text, then let an LLM write the answer using only that text - is called **Retrieval-Augmented Generation (RAG)**.

What makes this project different from a typical RAG tutorial project is the retrieval step. Most simple RAG systems use only one retrieval method (usually vector/semantic search) or blend two methods (keyword search + vector search) with a fixed, hand-picked ratio such as 50% keyword / 50% semantic for every query. This project instead trains a small machine learning model that looks at the current query and the two retrieval results, and predicts a custom weight for that specific query. That is the project's core research contribution.

1.2 What real-world problem does it solve?

Two retrieval techniques are commonly combined in modern search and RAG systems:

- **BM25 (lexical/keyword search):** good when the answer literally contains the same words as the question (e.g. exact product codes, names, numbers).
- **Dense/semantic search:** good when the question is phrased differently from the source text but means the same thing (paraphrases, synonyms, conceptual questions).

A fixed blend of the two (for example, always 50/50) is a compromise that is never perfectly correct: some queries are clearly keyword-heavy and would do better with a higher BM25 weight, while others are clearly conceptual and would do better with a higher dense weight. The project's README states this directly as the motivating problem, and frames the solution as learning `predicted_bm25_weight = f(query_features, retrieval_features)` instead of assuming the weight is always 0.5.

1.3 Why does this problem matter?

Retrieval quality is the ceiling for RAG answer quality - if the wrong text chunks are retrieved, the LLM cannot produce a correct grounded answer no matter how good the LLM is. Even a small, consistent improvement in retrieval precision/recall can materially improve the reliability of an LLM application. This matters in real deployments such as internal knowledge-base assistants, customer-support bots, and research assistants, where wrong retrieval leads directly to hallucinated or unhelpful answers.

1.4 What is the objective of the project?

- Build a working end-to-end RAG pipeline: document ingestion, retrieval, fusion, reranking, and grounded LLM generation.
- Replace the fixed-weight hybrid fusion step with a learned, per-query adaptive weight predictor.
- Provide a complete user-facing product around this pipeline: accounts, document upload, chat history, and a research/analytics dashboard showing retrieval behaviour.
- Evaluate the adaptive approach honestly against a fixed-weight baseline using standard information-retrieval metrics on a 1,000-query, multi-domain benchmark.

1.5 Who would use it?

As shipped, this is a research/portfolio project rather than a production product with a named customer. Two realistic user groups the design supports are: (1) a person who uploads their own PDF/DOCX/TXT files and chats with them (the 'user RAG' feature), and (2) a researcher/evaluator who wants to inspect retrieval behaviour, adaptive weights, and benchmark metrics through the dashboard.

HONESTY NOTE: The uploaded files do not contain evidence of real end users, deployment to a live server, or usage metrics from actual people. This should be described as a research/learning project, not a live product with users.

1.6 What are its major features?

- BM25 lexical retrieval and dense semantic retrieval, run in parallel over a large pre-indexed corpus.
- Adaptive BM25/Dense fusion, driven by a trained ExtraTreesRegressor model ("V4 Rich").
- Cross-encoder reranking of the fused candidates before they reach the LLM.
- User document upload (PDF, DOCX, TXT) with per-user document indexing and retrieval.
- Conversational chat with persistent conversation/message history.
- Full authentication: registration, login, sessions, email verification tokens, password reset tokens.
- Per-stage latency measurement (retrieval, fusion, reranking, prompt building, generation).
- A research/analytics dashboard (adaptive weights, retrieval metrics, recent queries).
- A 1,000-query, 4-domain (Argumentation, Finance, Medical, Scientific) evaluation benchmark comparing BM25-only, Dense-only, Fixed Hybrid, and Adaptive Hybrid retrieval.

1.7 What is the complete workflow (main corpus flow)?

```
User types a question in the React chat UI
|
v
POST /ask { "query": "..."} (FastAPI)
|
v
hybrid_search(query) [services/hybrid_service.py]
|-- dense_search() -> ChromaDB top-50 (all-MiniLM-L6-v2 embeddings)
|-- bm25_search() -> BM25Okapi top-50 (rank_bm25, persisted index)
|
v
retrieval_feature_service.extract(...) -> 17 numeric features
|
v
adaptive_predictor_v4.predict(...) -> ExtraTreesRegressor
| (predicted BM25 weight)
v
fusion_service.fuse(...) -> min-max normalize both
signals, weighted sum,
sort, top-10
|
v
reranker.rerank(...) -> cross-encoder re-scores
top candidates, top-5
|
v
prompt_builder.build_prompt(...) -> grounded-QA prompt with
numbered document chunks
|
```

```

v
llm_service.generate(prompt) -> Ollama -> Qwen2.5:7B
|
v
JSON response: answer + weights + documents + latency
|
v
React UI renders the answer, sources, and the
"Research Dashboard" (adaptive weights, pipeline steps, latency)

```

1.8 What happens from user input to final output (user-document flow)?

When the user is asking about their own uploaded document instead of the shared corpus, a parallel pipeline runs: the uploaded file is extracted to plain text, split into overlapping chunks, embedded, and stored in a Chroma collection scoped to that specific user (user_<user_id>-style collection). Answering a question then follows the same BM25 + Dense + adaptive-fusion + rerank pattern, but scoped only to that user's documents, plus an extra **context expansion** step that pulls in neighbouring chunks (by chunk_index) so the LLM doesn't see isolated, disconnected fragments.

1.9 What technologies are used?

Confirmed by reading backend/requirements.txt, imports in the code, and frontend/package.json. A full explained breakdown is in Part 2 and Part 4.

- Backend: Python 3, FastAPI, Uvicorn, Pydantic / pydantic-settings, SQLite (via the standard library sqlite3 module).
- Retrieval: ChromaDB (vector database), rank-bm25 (BM25Okapi), Sentence-Transformers (embeddings + cross-encoder reranker).
- ML: scikit-learn (ExtraTreesRegressor / RandomForestRegressor), NumPy, pandas, joblib (model persistence).
- LLM: Ollama Python client, running the Qwen2.5:7B model locally (no external API key used).
- Document parsing: pypdf, python-docx, PyMuPDF (listed in dependencies).
- Frontend: React 19, Vite 8 (dev server/bundler), lucide-react (icon set), plain hand-written CSS.

1.10 Why is each technology used?

Short version here; the full WHAT/WHY/HOW/WHERE/Interview-Questions treatment for every technology is in Part 4. In brief: FastAPI gives fast, typed, auto-documented REST APIs with minimal boilerplate; ChromaDB gives a ready-made, persistent vector store with a simple query API so the project doesn't need to implement approximate nearest-neighbour search by hand; rank-bm25 gives a battle-tested BM25 implementation; Sentence-Transformers gives pretrained embedding and cross-encoder models without training them from scratch; scikit-learn's tree ensembles are a strong, fast-to-train, interpretable choice for a small tabular regression problem (predicting one weight from 17 numeric features) where a deep neural network would be overkill; Ollama + Qwen2.5:7B keeps generation fully local and free of per-token API costs; React + Vite gives a fast, component-based frontend development experience.

1.11 What is the overall architecture?

The system is a classic three-tier web application with an added ML/retrieval subsystem: a React single-page frontend talks over HTTP/JSON to a FastAPI backend, which in turn talks to three kinds of storage/compute: (1) SQLite for user accounts and conversations, (2) ChromaDB for vector storage and similarity search, and (3) a locally running Ollama server for LLM text generation. A full diagram is in Part 11.

1.12 How do different components communicate?

- Frontend <-> Backend: HTTP requests (fetch API) carrying JSON, with a Bearer token in the Authorization header for authenticated routes.
- Backend <-> SQLite: direct SQL via Python's built-in `sqlite3` module (no ORM).
- Backend <-> ChromaDB: the `chromadb.PersistentClient` Python API (in-process, reads/writes a local folder, not a separate network service in this deployment).
- Backend <-> Ollama: the `ollama` Python client, which talks to a locally running Ollama server (Ollama itself exposes a local HTTP API on the machine).
- Backend internal: plain Python function calls between service modules (no message queue or internal network hop - it is a single Python process, not microservices).

1.13 What is the role of each major component?

api/	FastAPI routers - defines every HTTP endpoint (auth, chat, documents, conversations, analytics).
services/	Business logic - retrieval, fusion, reranking, prompting, LLM calls, auth logic, DB access.
services/user_rag/	The parallel retrieval pipeline scoped to a single user's uploaded documents.
services/ingestion/	Turns an uploaded file into clean text and then into overlapping text chunks.
services/training/	Offline scripts used to build training data and train the adaptive weight model.
evaluation/	Offline benchmark and metric scripts used to measure retrieval quality.
config/	Central place for constants such as TOP_K, CANDIDATE_K, the embedding model name, and paths.
frontend/src/App.jsx	The entire React UI: authentication screens, chat interface, and research dashboard.

1.14 What is the folder/file structure?

```
adaptive-hybrid-rag-main/
├── backend/
│   ├── api/ (FastAPI route definitions)
│   ├── config/ (settings.py - constants and paths)
│   ├── evaluation/ (benchmark + metrics scripts, results CSVs)
│   ├── services/ (all business logic, described above)
│   │   ├── ingestion/ (document_extractor.py, chunker.py)
│   │   ├── user_rag/ (user_indexer.py, user_retriever.py, user_pipeline.py)
│   │   ├── training/ (scripts that build training data & train the ML model)
│   │   ├── utils/ (small helpers)
│   └── app.py (FastAPI application entry point - THE real server)
```

- ■■■ main.py (a second, smaller, unused FastAPI app - see Part 19)
- ■■■ requirements.txt (actual backend dependency list)
- ■■■ sample_user_document*.txt (sample files for manually testing user RAG)
- ■■■ training_dataset.csv
- frontend/
- ■■■ src/App.jsx, App.css, main.jsx, index.css
- ■■■ package.json, vite.config.js
- data/raw/, data/processed/ (placeholders for source documents - mostly empty)
- docs/ (empty placeholder folders: thesis, ieee, literature, diagrams)
- requirements.txt (root-level - broader, partly unused list, see Part 19)
- README.md
- LICENSE (present but empty - see Part 19)

1.15 "PROJECT IN 30 SECONDS"

"Adaptive Hybrid RAG is a document question-answering system. Instead of always blending keyword search and semantic search 50/50, I trained a small machine learning model that looks at each query and predicts the best blend for that specific query. It then reranks the results with a cross-encoder and uses a local LLM, Qwen2.5, to generate a grounded answer from the retrieved text. I evaluated it on a 1,000-query benchmark and it improved precision, recall, and nDCG over a fixed 50/50 baseline."

1.16 "PROJECT IN 1 MINUTE"

"The core problem is that hybrid retrieval systems usually combine BM25 keyword search and dense vector search using a fixed weight, but different queries actually favour different signals - keyword-heavy queries do better with BM25, paraphrased or conceptual queries do better with dense retrieval. I built a pipeline that runs both retrievers in parallel, extracts 17 features describing how confident and how much each retriever agrees (score gaps, overlap ratio, rank agreement, lexical coverage, and so on), and feeds those features into an ExtraTreesRegressor that predicts the right BM25/Dense weight for that query. The fused results are reranked with a cross-encoder and passed to a locally running Qwen2.5:7B model with a strict grounded-answer prompt. Around that pipeline I built a full product: user accounts with proper password hashing, document upload with per-user retrieval, persistent chat history, and a dashboard that shows the adaptive weights and latency for every query. I also built an honest 1,000-query, 4-domain evaluation that compares BM25-only, Dense-only, Fixed Hybrid, and Adaptive Hybrid retrieval using standard IR metrics."

1.17 "PROJECT IN 2-3 MINUTES"

"The motivation comes from a known limitation in hybrid retrieval: most systems fuse BM25 and dense vector search scores using a single fixed weight for every query, even though the best weight genuinely differs by query type. My project addresses this with a learned, per-query adaptive fusion weight.

For every query, I run BM25 (using the rank-bm25 library, over an index I persist to disk so it doesn't have to be rebuilt on every server start) and dense retrieval (Sentence-Transformers embeddings stored in ChromaDB), each returning the top 50 candidates. From those two result sets I extract 17 numeric features - things like the top BM25 score, the gap between the top and second score, how much the BM25 and dense result sets overlap, how well their rankings agree, and how many of the query's words literally appear in the top retrieved chunk. These features go into an ExtraTreesRegressor that I trained to predict a BM25 fusion weight. I didn't hand-label these training targets - I generated them by actually sweeping many possible BM25/Dense weight combinations per query and keeping whichever weight produced the best retrieval quality for that query, which then became the ground-truth label.

Once the adaptive weight is predicted, I independently min-max normalize the BM25 and dense scores and combine them with that weight, matching documents across the two result sets using a stable

document/chunk ID rather than exact text matching. The fused top results then go through a cross-encoder reranker for a more accurate final ordering, and the top few chunks are placed into a strict grounded-answer prompt that explicitly tells the LLM - a locally running Qwen2.5:7B served through Ollama - not to use outside knowledge.

Around this retrieval core I built the rest of the application: FastAPI backend with SQLite-backed authentication (PBKDF2 password hashing with per-user salts, session tokens, email-verification and password-reset flows), persistent conversation history, and a parallel 'user document RAG' path so a person can upload their own PDF, DOCX, or TXT file, have it chunked and embedded into a private collection, and ask questions about only that document. The React frontend provides the chat UI plus a research dashboard that shows the live adaptive weights and per-stage latency for each answer.

For evaluation, I built a 1,000-query benchmark spanning four domains and compared BM25-only, Dense-only, Fixed Hybrid, and Adaptive Hybrid retrieval using Precision@10, Recall@10, MRR, and nDCG@10, which I implemented myself rather than importing from a library. The adaptive method improved Precision@10, Recall@10, and nDCG@10, but MRR was very slightly lower, so I report that honestly instead of only highlighting the metrics that improved."

Full Technology Inventory (What Is Actually Used)

This inventory is built only from what appears in `backend/requirements.txt`, actual import statements in the code, and `frontend/package.json`. Nothing here is assumed just because it is common for RAG projects.

Languages

- Python 3 (backend, all services, training, evaluation).
- JavaScript / JSX (frontend, React components).
- SQL (raw SQL strings inside `sqlite3` calls - no ORM is used).
- CSS (hand-written, in `App.css` / `index.css` - no CSS framework such as Tailwind or Bootstrap is used).

Backend framework and server

- FastAPI - the web framework (`fastapi==0.140.13`).
- Uvicorn - the ASGI server that actually runs FastAPI (`uvicorn==0.51.0`).
- Pydantic / pydantic-settings - request/response validation and settings management.
- python-multipart - required by FastAPI to parse multipart/form-data (file uploads).

Database

- SQLite (Python standard library `sqlite3` module) - two separate database files: `app.db` for authentication and `rag_app.db` for conversations.
- ChromaDB (`chromadb==1.5.9`) - the vector database used for both the shared corpus and each user's private document collection.

Retrieval

- rank-bm25 (`rank-bm25==0.2.2`) - BM25Okapi implementation for lexical/keyword retrieval.
- Sentence-Transformers (`sentence-transformers==3.0.1` in `backend/requirements.txt`) - used for both the embedding model (all-MiniLM-L6-v2) and the cross-encoder reranker (ms-marco-MiniLM-L-6-v2).
- PyTorch (`torch==2.6.0`) - the deep-learning backend that Sentence-Transformers runs on.

Machine learning (adaptive weight predictor)

- scikit-learn (`scikit-learn==1.9.0`) - `ExtraTreesRegressor` and `RandomForestRegressor`, train/test split, MAE / RMSE / R^2 metrics.
- NumPy and pandas - numeric arrays and tabular feature data.
- joblib - saving and loading the trained model (.pkl file).

LLM / generation

- ollama (`ollama==0.6.0`) Python client - talks to a locally running Ollama server.
- Model used: `qwen2.5:7b` (confirmed in `services/llm_service.py` and the `/info` endpoint in `app.py`).
- No external LLM API (OpenAI, Anthropic, etc.) is used anywhere in the code - generation is fully local.

Document parsing

- pypdf - extracts text from PDF files, page by page.
- python-docx (imported as docx) - extracts paragraph text from DOCX files.
- PyMuPDF (listed in requirements, used by the older/legacy pdf_service.py prototype).

Frontend

- React 19 (react, react-dom) - component-based UI library.
- Vite 8 - development server and production build tool.
- lucide-react - icon component library.
- No state-management library (no Redux/Zustand) - state is handled with React's built-in useState/useEffect hooks.
- No CSS framework - styling is hand-written CSS (App.css is 3,179 lines).

Authentication / Security

- hashlib.pbkdf2_hmac (Python standard library) - password hashing, SHA-256, 120,000 iterations, per-user salt.
- secrets (Python standard library) - cryptographically secure random tokens for session IDs, verification tokens, and password-reset tokens.
- hmac.compare_digest - constant-time comparison when verifying password hashes (prevents timing attacks).
- No JWT library is used - sessions are simple random opaque tokens stored in a SQLite table, not self-contained signed tokens.

Algorithms actually present

- BM25 (Okapi BM25) ranking function - via rank-bm25.
- Approximate/exact nearest-neighbour vector search - via ChromaDB (implementation detail is internal to Chroma).
- ExtraTreesRegressor - an ensemble of randomized decision trees, used for the adaptive weight prediction.
- Min-max normalization - used to make BM25 and dense scores comparable before fusion.
- PBKDF2 (Password-Based Key Derivation Function 2) - for password hashing.
- Standard IR ranking metrics implemented by hand: Precision@K, Recall@K, Reciprocal Rank, DCG@K, nDCG@K (evaluation/metrics.py) - not imported from a library such as scikit-learn or ranx.

Data structures actually present

- Python dict used as a hash map to join/merge BM25 and dense result sets by a stable document key (services/fusion_service.py).
- Python set used for overlap/Jaccard-style calculations between BM25 and dense result ID sets (services/retrieval_feature_service.py).
- NumPy arrays for all numeric score vectors (fast vectorized statistics: mean, std, argsort).
- SQLite tables (relational rows) for users, sessions, tokens, conversations, and messages.
- An inverted index, built and owned internally by the rank-bm25 library (not implemented by hand in this project).

Machine learning / LLM / RAG components actually present

- Yes - this project is fundamentally a RAG project with an ML component (the adaptive weight predictor). A full breakdown is in Part 10.

Deployment configuration

HONESTY NOTE: No Dockerfile, docker-compose.yml, CI/CD configuration (e.g. GitHub Actions), or cloud deployment configuration (Procfile, Render/Railway/Vercel config, Kubernetes manifests) was found anywhere in the uploaded ZIP. The README's installation instructions describe only local development (Uvicorn on localhost:8000, Vite dev server). This should be described honestly as a locally-run project, not a deployed/production system, unless you have personally deployed it elsewhere.

Testing

HONESTY NOTE: No automated test suite (no pytest files, no test_*.py under a conventional tests/ directory, no frontend testing library) was found. There ARE files named things like services/test_vector_db.py and services/test_logging_service.py, but based on their location and naming they read as manual/ad-hoc verification scripts rather than an automated pytest suite with assertions run in CI. The evaluation/ scripts are research benchmarks (measuring retrieval quality), not software correctness tests. If asked about testing, be honest that formal unit/integration testing was not a focus of this project.

Configuration and environment variables

- config/settings.py centralizes constants: PROJECT_ROOT, CHROMA_DB_DIR, COLLECTION_NAME, EMBEDDING_MODEL, TOP_K=10, CANDIDATE_K=50.
- config.py at the backend root exists but is an empty file (0 lines) - config/settings.py is the one actually used.

HONESTY NOTE: No .env file or python-dotenv usage was found in the backend code search, even though python-dotenv appears in the root requirements.txt. Configuration in this project is done with plain Python constants, not environment variables loaded from a .env file.

Build tools / package managers

- pip - Python dependency installation (requirements.txt files).
- npm - frontend dependency installation and script running (package.json, package-lock.json).
- Vite - frontend bundling/dev server (not Webpack or Create React App).

Version control

The README references a GitHub repository URL (github.com/Parasfunde21/adaptive-hybrid-rag) and the project includes a .gitignore file that excludes local runtime data such as the SQLite databases, the ChromaDB folder, and the BM25 cache - so Git is clearly the version control system used.

File-by-File Explanation of Key Files

The project has well over 60 Python files. This section explains the files that matter most for an interview - the ones that carry the project's actual logic - rather than every trivial file.

3.1 backend/app.py (the real FastAPI entry point)

Purpose	Creates and configures the FastAPI application object that Uvicorn actually runs.
Why it exists	Every FastAPI project needs one place that builds the `app` object, wires in middleware, and includes the routers.
Important lines	<code>`app = FastAPI(...)`</code> , CORS middleware restricted to <code>`http://localhost:5173` / `127.0.0.1:5173`</code> (the Vite dev server), <code>`app.include_router(router)`</code> , a global exception handler, and three simple routes: <code>`/`</code> (health message), <code>`/health`</code> , and <code>`/info`</code> (returns project metadata: LLM name, embedding model, reranker, fusion strategy).
Inputs	None directly - it wires together the `router` object built in <code>api/routes.py</code> .
Outputs	The configured FastAPI `app` object that Uvicorn serves on port 8000.
Dependencies	<code>api.routes</code> , <code>api.exceptions</code> , <code>fastapi</code> , <code>fastapi.middleware.cors</code> .
Connects to	Every API route file indirectly, through <code>api/routes.py</code> .
Possible interview questions	Why did you restrict CORS to specific origins instead of `*`? What does <code>`allow_credentials=True`</code> mean, and why is it needed for cookie/token-based auth? What does the <code>`/info`</code> endpoint tell a client, and why might an endpoint like this be useful?

3.2 backend/services/hybrid_service.py (the retrieval orchestrator)

Purpose	Coordinates the entire adaptive hybrid retrieval process for one query, end to end.
Why it exists	Retrieval needs several steps done in a specific order (retrieve, extract features, predict weight, fuse, attach citation info, log). This class is the single place that owns that order.
Important class/functions	<code>`AdaptiveHybridRetriever.search()`</code> , exposed publicly as <code>`hybrid_search()`</code> .
Important logic	Calls <code>`dense_search()`</code> and <code>`bm25_search()`</code> for <code>CANDIDATE_K=50</code> candidates each, builds 17 features via <code>`retrieval_feature_service.extract()`</code> , gets a predicted weight from <code>`adaptive_predictor_v4.predict()`</code> , fuses with <code>`fusion_service.fuse()`</code> , keeps the final <code>TOP_K=10</code> , and attaches <code>`source`</code> , <code>`document_id`</code> , <code>`chunk_id`</code> , and a <code>`citation`</code> dict to every result for traceability.
Inputs	<code>`query: str`</code> , <code>`top_k: int`</code> (defaults to <code>TOP_K=10</code>).

Outputs	A dict with `query`, `weights` (bm25/dense), `model` name, `retrieval_config`, `latency`, and `results` (the ranked, citation-tagged documents).
Dependencies	retrieval_service, bm25_service, adaptive_predictor_v4, retrieval_feature_service, fusion_service, logging_service, config.settings.
Connects to	Called by services/rag_pipeline.py (the shared corpus) and by api/routes.py's `/ask` endpoint indirectly through rag_pipeline.
Possible interview questions	Why fetch 50 candidates from each retriever but only keep 10 at the end? What would happen to retrieval quality if CANDIDATE_K were reduced to 10? Walk me through what happens if BM25 returns zero results for a query.

3.3 backend/services/fusion_service.py (AdaptiveScoreFusion)

Purpose	Combines BM25 and dense retrieval results into a single ranked list using the adaptive weights.
Why it exists	BM25 scores and dense distances live on completely different numeric scales, so they cannot be added together directly - they must be normalized first, and then documents that were found by only one retriever must still be handled correctly.
Important functions	<code>`min_max_normalize()`</code> , <code>`distance_to_similarity()`</code> ($1 / (1 + \text{distance})$), <code>`_document_key()`</code> (chooses chunk_id, then document_id, then retriever ID, then a text hash, as the join key), and <code>`fuse()`</code> (the main entry point).
Important logic	For every candidate found by either retriever, a result dict is created or updated (so a document found by both retrievers merges into ONE row instead of two). <code>`fusion_score = dense_weight * dense_component + bm25_weight * bm25_component`</code> , then the merged list is sorted descending by <code>`fusion_score`</code> .
Inputs	Both retrievers' documents, distances/scores, ids, metadatas, plus <code>`dense_weight`</code> and <code>`bm25_weight`</code> from the adaptive predictor.
Outputs	A ranked list of dicts, each carrying both raw and normalized scores, which retriever(s) found it, and its rank in each individual retriever.
Dependencies	numpy only.
Connects to	Called from hybrid_service.py and services/user_rag/user_retriever.py.
Possible interview questions	Why min-max normalize instead of using raw scores directly? What happens if every BM25 score is identical (e.g. all zero)? Why prefer a stable ID over matching by document text?

3.4 backend/services/adaptive_predictor_v4.py (the trained ML model wrapper)

Purpose	Loads the trained ExtraTreesRegressor model once at startup and exposes a <code>`predict()`</code> method.
----------------	--

Why it exists	Keeps model-loading and prediction logic in one place, separate from where the features are computed and separate from how the weight is used.
Important logic	Loads <code>`models/adaptive_weight_model_v4_rich.pkl`</code> with <code>`joblib`</code> ; builds a single-row pandas DataFrame with the 17 features in a fixed column order (order matters for scikit-learn models); clips the prediction to the valid range <code>[0, 1]</code> with <code>`np.clip`</code> ; returns both <code>`bm25_weight`</code> and <code>`dense_weight = 1 - bm25_weight`</code> (they are guaranteed to sum to 1 by construction, not learned independently).
Inputs	<code>`query: str`, `retrieval_features: dict`</code> (the 17 features).
Outputs	<code>`{"bm25_weight": float, "dense_weight": float, "model": "Extra Trees V4 Rich"}`</code> .
Dependencies	joblib, numpy, pandas, os.
Connects to	Called by <code>hybrid_service.py</code> after feature extraction, before fusion.
Possible interview questions	Why does the constructor raise <code>`FileNotFoundError`</code> if the model is missing, instead of falling back to a fixed weight? Why is the prediction clipped to <code>[0, 1]</code> ? Why is the model loaded once as a module-level singleton instead of on every request?

3.5 backend/services/retrieval_feature_service.py (feature engineering)

Purpose	Turns the raw output of BM25 and dense retrieval into the 17 numeric features the ML model needs.
Why it exists	An ML model cannot consume raw text or ranked lists directly - it needs fixed-length numeric input. This class defines exactly how retrieval behaviour is summarized into numbers.
Important logic	Computes top/mean/std/gap/top-mean-ratio for both BM25 scores and dense similarities (<code>`1/(1+distance)`</code>); computes <code>`overlap_ratio`</code> (Jaccard similarity between the BM25 and dense document ID sets); computes <code>`rank_agreement`</code> (1 minus the average absolute rank difference for documents both retrievers found, normalized); computes query-level stats (token count, unique token count, average token length); computes lexical coverage (fraction of query tokens that literally appear in the top retrieved chunk, for both BM25 and dense).
Notable engineering detail	The <code>`_resolve_arguments()`</code> method deliberately supports FOUR different calling conventions (legacy 4-positional-argument V3 calls, 5-positional V4 calls, pure keyword calls, and mixed calls), so both the old V3 code paths and the new V4 code paths can call the same method without breaking. This is a form of backward-compatible API design.
Inputs	Query string, BM25 scores, dense distances, BM25 doc texts, dense doc texts (various calling conventions).
Outputs	A dict of 17 named V4 features, plus a few extra legacy-named keys for V3 compatibility.
Dependencies	re (regex tokenizer), numpy.

Possible interview questions	Why maintain 4 calling conventions instead of just breaking the old code and fixing every call site? What does <code>`rank_agreement`</code> actually measure, in plain English? Why is the regex <code>`\b[w+#]+\b`</code> used for tokenizing instead of a simple <code>`split()`</code> ?
-------------------------------------	--

3.6 backend/services/bm25_service.py (persistent BM25 index)

Purpose	Builds and serves BM25 lexical search over the entire ChromaDB corpus (75,000+ documents), and avoids rebuilding the index on every server restart.
Why it exists	BM25Okapi has to tokenize and index the whole corpus before it can answer any query; for tens of thousands of documents this is slow, so the index is cached to disk with pickle.
Important logic	<code>`_collection_fingerprint()`</code> samples 3 document IDs (first, middle, last) from ChromaDB and hashes them together with the collection name and count with SHA-256, instead of reading the whole collection - a lightweight way to detect "has the underlying data changed?" without an expensive full scan. <code>`_load_cache()`</code> validates the cache version, collection name, document count, AND fingerprint before trusting a cached index. <code>`_save_cache()`</code> writes to a temporary file first and then uses <code>`os.replace()`</code> to atomically swap it in, so a crash mid-write can't corrupt the cache.
Inputs	Query string, <code>`top_k`</code> .
Outputs	Tuple of (documents, scores, ids, metadatas) for the top-k BM25 matches.
Dependencies	chromadb, rank_bm25, numpy, pickle, hashlib.
Possible interview questions	Why sample only 3 documents for the fingerprint instead of hashing the whole corpus? What could go wrong with that approach (edge case: middle documents change but sampled ones don't)? Why write to a temp file and then <code>`os.replace()`</code> instead of writing the cache file directly?

3.7 backend/services/auth_service.py (authentication + SQLite)

Purpose	All authentication logic and all direct SQL for the <code>`app.db`</code> database (users, sessions, email verification, password reset).
Why it exists	Keeps password hashing, session handling, and the SQL schema in a single, auditable file instead of scattering raw SQL across API route handlers.
Important functions	<code>`hash_password()`</code> / <code>`verify_password()`</code> (PBKDF2-HMAC-SHA256, 120,000 iterations, random 16-byte salt per user, <code>`hmac.compare_digest`</code> for timing-safe comparison), <code>`create_user()`</code> , <code>`authenticate_user()`</code> , <code>`create_session()`</code> / <code>`get_user_from_token()`</code> / <code>`delete_session()`</code> , <code>`create_verification_token()`</code> / <code>`verify_email()`</code> , <code>`create_password_reset_token()`</code> / <code>`reset_password()`</code> .
Important logic	On password reset, ALL of that user's existing sessions are deleted (<code>`DELETE FROM sessions WHERE user_id = ?`</code>) - a real security practice: if a password is compromised and then changed, old stolen session tokens are invalidated immediately.

Inputs/Outputs	Plain Python function calls with primitive arguments in, dicts/booleans/tokens out.
Dependencies	hashlib, hmac, secrets, sqlite3, pathlib, datetime - all Python standard library, no external auth package.
Possible interview questions	Why PBKDF2 instead of plain SHA-256 of the password? Why does every user get a different salt? What attack does <code>hmac.compare_digest</code> specifically prevent, and why does it matter here? Why invalidate sessions on password reset?

3.8 backend/services/ingestion/chunker.py (DocumentChunker)

Purpose	Splits extracted document text into overlapping fixed-size chunks before embedding.
Why it exists	Embedding models and the LLM's context window can't handle an entire document as one unit; documents must be split into smaller pieces that can each be individually retrieved.
Important logic	A sliding window: <code>chunk_size=500</code> characters, <code>overlap=100</code> characters. Starting at index 0, it slices <code>text[start:end]</code> , then moves <code>start</code> forward by <code>chunk_size - overlap</code> (implemented as <code>start = end - overlap</code>), so consecutive chunks share the last 100 characters of the previous chunk. This prevents a sentence or idea from being cut exactly at a chunk boundary and lost.
Inputs	Raw extracted document text (a single string).
Outputs	A list of overlapping text chunks (strings).
Dependencies	None - pure Python string slicing.
Possible interview questions	Why use character-based chunking instead of splitting by sentence or paragraph? What is the effect of increasing the overlap? What happens for a document shorter than 500 characters?

3.9 backend/services/user_rag/ (user_indexer.py, user_retriever.py, user_pipeline.py)

This package is a parallel version of the shared-corpus pipeline, scoped to one user's own uploaded documents. `user_indexer.py` extracts text, chunks it, embeds it, and writes it into a Chroma collection that belongs only to that user. `user_retriever.py` (`UserAdaptiveRetriever`) runs the same BM25 + dense + adaptive-fusion pattern as the shared pipeline, but reads from that per-user collection. `user_pipeline.py` (`UserRAGPipeline.answer()`) adds an extra step not present in the shared pipeline: **context expansion** - after reranking, it looks up neighbouring chunks (by `chunk_index` within the same `document_id/file_id`) so the LLM receives a more complete, connected piece of context rather than an isolated 500-character fragment.

IMPORTANT: This is a genuinely good detail to mention in an interview: it shows you noticed that top-ranked chunks in isolation can be incomplete, and you designed a specific fix (neighbour expansion) for it - this is exactly the kind of practical RAG engineering interviewers like to hear about.

3.10 frontend/src/App.jsx (the entire React application)

Purpose	The single file that implements the whole frontend: login/register screens, the chat UI, document upload, conversation sidebar, and the research dashboard.
Why it is one large file	The project does not split components into separate files under `src/components/` - every component (`AuthScreen`, `MetricCard`, `PipelineStep`, `ResearchDashboard`, `App`, and several small display components) is defined in this one 2,318-line file.
Important logic	`apiRequest()` is a shared fetch wrapper that automatically attaches the Bearer token from `localStorage` and throws a normal JavaScript Error on a non-OK HTTP response, so every component can `await apiRequest(...)` inside a `try/catch`. `getStoredToken()`/`getStoredUser()` read persisted login state from `localStorage` on page load so a refresh doesn't log the user out.
Input handling	Controlled `` elements backed by `useState`, submitted on form `onSubmit`/button `onClick` handlers.
Dependencies	react, lucide-react.
Possible interview questions	Why store the auth token in `localStorage` instead of a cookie, and what is the security trade-off (XSS risk vs CSRF risk)? Why centralize all API calls in one `apiRequest` helper instead of calling `fetch` directly everywhere? What would you refactor first if this file kept growing?

HONESTY NOTE: If asked why App.jsx is one 2,318-line file instead of split into a components/ folder, the honest answer is that it is a pragmatic choice made to move fast on a research project, not an architectural best practice - and a real improvement you can name is splitting it into logical components (AuthScreen, ChatWindow, Sidebar, ResearchDashboard) as separate files.

Every Technology Explained From Zero

Format for each technology: **WHAT -> WHY -> HOW -> WHERE USED IN MY PROJECT -> INTERVIEW QUESTIONS.**

4.1 Python

WHAT: A high-level, interpreted, dynamically-typed general-purpose programming language.

WHY: Rich ecosystem for both web backends (FastAPI) and machine learning/NLP (scikit-learn, Sentence-Transformers, PyTorch) - almost no other language has both as first-class citizens.

HOW (basic mechanics): Code is parsed and compiled to bytecode, run by the CPython interpreter (or another runtime). Uses indentation for blocks, dynamic typing, and automatic memory management (garbage collection).

WHERE USED IN MY PROJECT: The entire backend - API layer, retrieval, fusion, ML training, evaluation - is Python.

Interview questions this could prompt:

- What is the difference between a list and a tuple in Python?
- How does Python manage memory / garbage collection?
- What is the Global Interpreter Lock (GIL) and why does it matter for this project's use of threads?

4.2 FastAPI

WHAT: A modern Python web framework for building REST APIs, built on top of Starlette (ASGI) and Pydantic.

WHY: Gives automatic request/response validation from type hints, automatic OpenAPI/Swagger documentation, and native async support, with much less boilerplate than Flask or Django REST Framework.

HOW (basic mechanics): Routes are defined with decorators like `@router.post('/ask')`. A function's type-hinted parameters (e.g. `request: QueryRequest`) are automatically parsed and validated from the incoming JSON body by Pydantic; the return value is automatically serialized back to JSON.

WHERE USED IN MY PROJECT: Every backend endpoint: `/ask`, `/auth/*`, `/user/*`, `/conversations/*`, `/analytics/*`. `app.py` builds the FastAPI app and mounts a combined `router` from `api/routes.py`.

Interview questions this could prompt:

- What is ASGI and how is it different from WSGI (used by Flask)?
- How does FastAPI generate automatic API documentation?
- How would you add rate limiting or authentication middleware globally instead of per-route?

4.3 Pydantic

WHAT: A Python data-validation library that uses type hints to define and validate data schemas.

WHY: Ensures the API only accepts well-formed requests (e.g. rejects a login request missing a password) and produces consistent, typed JSON responses.

HOW (basic mechanics): You declare a class inheriting `BaseModel` with typed fields (optionally with `Field(min_length=..., max_length=...)` constraints); Pydantic validates incoming data against it and raises a 422 error automatically on failure.

WHERE USED IN MY PROJECT: `api/schemas.py` (`QueryRequest`, `QueryResponse`, `RetrievedDocument`, etc.) and inline schema classes inside `api/auth.py` (`RegisterRequest`, `LoginRequest`, etc.).

Interview questions this could prompt:

- What happens if a client sends a field with the wrong type?
- What does ``class Config: extra = "allow"``` do on ``QueryResponse``, and why might that be useful here?
- How is Pydantic v2 different from v1 (performance/architecture)?

4.4 SQLite

WHAT: A lightweight, file-based, serverless relational (SQL) database engine built into a single file on disk.

WHY: Needs no separate database server to install or run, which is ideal for a project developed and run on one machine; it still gives full SQL, transactions, and foreign keys.

HOW (basic mechanics): The whole database (tables, indexes, data) lives in one ``.db`` file. Python's standard library ``sqlite3`` module opens a connection directly to that file and executes SQL strings.

WHERE USED IN MY PROJECT: Two database files: ``.backend/data/app.db`` (users, sessions, email_verification_tokens, password_reset_tokens - built by ``auth_service.py``) and ``.backend/data/rag_app.db`` (conversations, messages - built by ``conversation_service.py``).

Interview questions this could prompt:

- Why two separate SQLite files instead of one database with more tables?
- What are the limits of SQLite for concurrent writes, and would it scale to many simultaneous users?
- What would you change to move this to PostgreSQL in production?

4.5 ChromaDB (Vector Database)

WHAT: An open-source database specialized for storing high-dimensional vectors (embeddings) and performing fast similarity search over them.

WHY: Dense/semantic retrieval requires comparing a query's embedding against potentially tens of thousands of document embeddings; a vector database makes that fast and persistent, instead of hand-rolling brute-force cosine similarity in a loop.

HOW (basic mechanics): Documents are inserted with their embedding vector, raw text, an ID, and metadata. A query embedding is then compared against all stored vectors and the closest ones (lowest distance) are returned. ``chromadb.PersistentClient`` stores everything on disk so it survives a restart.

WHERE USED IN MY PROJECT: ``services/retrieval_service.py`` (shared corpus, collection ``adaptive_rag_beir_v2``) and ``services/user_rag/user_indexer.py`` / ``user_retriever.py`` (one private collection per user).

Interview questions this could prompt:

- What distance metric does the collection use, and how does that affect the similarity conversion ``1/(1+distance)`` used in this project?
- How is a vector database different from a normal SQL database?
- How would ChromaDB behave differently from an approximate-nearest-neighbour index like FAISS or HNSW at very large scale?

4.6 BM25 (via rank-bm25)

WHAT: A classic lexical ranking function ("Best Matching 25") that scores how relevant a document is to a query based on term frequency, inverse document frequency, and document length normalization.

WHY: Dense/semantic search can miss queries that need an exact keyword or code match; BM25 is strong at exactly this and is the natural complement to dense retrieval in a hybrid system.

HOW (basic mechanics): Each document is tokenized (lowercased, split on whitespace in this project) into a term list; BM25Okapi builds statistics over the whole corpus (term frequencies, document lengths, average document length); ``get_scores(query_tokens)`` returns a relevance score for every document, and the top-k by score are kept.

WHERE USED IN MY PROJECT: ``services/bm25_service.py`` builds and serves a persistent BM25Okapi index over the entire ChromaDB corpus; the per-user pipeline rebuilds a small BM25 index over just that user's chunks.

Interview questions this could prompt:

- Explain BM25's formula conceptually - what role does term frequency saturation play, and why does BM25 avoid rewarding a term that simply appears many times, unlike raw TF-IDF?
- Why does the code just call ``lower().split()`` for tokenization instead of a proper NLP tokenizer - what are the downsides of this simple approach?
- What happens to BM25 scores for a query with a word that never appears in the corpus?

4.7 Sentence-Transformers (Embedding Model: all-MiniLM-L6-v2)

WHAT: A Python library, built on PyTorch and Hugging Face Transformers, that produces dense vector embeddings for sentences/paragraphs such that semantically similar text has vectors that are close together.

WHY: Semantic retrieval needs a way to turn text into vectors; this is the standard tool for that, and all-MiniLM-L6-v2 is a small, fast, widely-used general-purpose embedding model, well suited for a project that needs to embed tens of thousands of chunks and run locally without a GPU cluster.

HOW (basic mechanics): A sentence goes through a transformer encoder (a distilled/compressed MiniLM model) and the token representations are pooled (typically mean-pooled) into one fixed-size (384-dimensional for this model) vector representing the whole input.

WHERE USED IN MY PROJECT: ``services/model_loader.py`` loads the model once at startup, strictly from local cache (``HF_HUB_OFFLINE=1``); ``services/retrieval_service.py`` calls ``encode(query)`` to embed the incoming query before querying ChromaDB; the same model embeds every document chunk when the corpus/user documents were originally indexed.

Interview questions this could prompt:

- Why load the model with ``local_files_only=True`` and offline environment variables instead of just calling ``SentenceTransformer(name)`` normally?
- What does it mean for embeddings to be 'normalized' or not, and why does this project pass ``normalize_embeddings=False``?
- Why choose a small model like MiniLM instead of a larger, more accurate embedding model?

4.8 Cross-Encoder Reranking (ms-marco-MiniLM-L-6-v2)

WHAT: A transformer model that takes a (query, document) PAIR as joint input and outputs a single relevance score, as opposed to a bi-encoder/embedding model that encodes query and document separately.

WHY: Cross-encoders are more accurate at judging relevance than embedding similarity alone (because the model can directly attend across both texts), but far too slow to run over an entire corpus - so they are used only to re-score a small shortlist of already-retrieved candidates.

HOW (basic mechanics): The model receives ``(query, document)`` as one input and is trained to predict a relevance score; higher score means more relevant. The candidates are then sorted by this new score.

WHERE USED IN MY PROJECT: ``services/reranker_service.py`` (``CrossEncoderReranker.rerank()``) - takes the fused hybrid results and re-scores/re-sorts the top candidates, keeping only the top 5 before they go into the LLM prompt.

Interview questions this could prompt:

- Why not use the cross-encoder for the initial retrieval over the whole corpus, if it's more accurate?
- What is the practical latency/cost trade-off of adding a reranking stage?
- What's the conceptual difference between a bi-encoder (used for the embeddings) and a cross-encoder (used for reranking)?

4.9 scikit-learn / ExtraTreesRegressor

WHAT: scikit-learn is a general-purpose Python machine learning library. ExtraTreesRegressor ('Extremely Randomized Trees') is an ensemble regression model made of many decision trees.

WHY: The adaptive-weight-prediction problem is a small tabular regression task (17 numeric features -> one number between 0 and 1); tree ensembles handle this kind of structured/tabular data very well, train fast, and don't need GPU or huge amounts of data, unlike a neural network.

HOW (basic mechanics): Many decision trees are trained on random subsets of features; unlike a standard Random Forest, Extra Trees also pick the SPLIT THRESHOLD randomly (rather than searching for the optimal split), which adds more randomness, typically reduces variance further, and trains faster. The final prediction is the average of all trees' predictions.

WHERE USED IN MY PROJECT: ``backend/training/train_v4_rich_model.py`` trains it (500 estimators, ``max_features=0.8``, ``min_samples_leaf=2``, ``random_state=42``) and compares it against a RandomForestRegressor. ``services/adaptive_predictor_v4.py`` loads the trained model and calls ``predict()`` at inference time.

Interview questions this could prompt:

- What's the difference between Extra Trees and Random Forest specifically?
- Why was Extra Trees chosen over Random Forest here - what does the comparison in the training script show?
- What does ``min_samples_leaf=2`` control, and what would happen if it were set to 1 or to 50?
- Why not use a neural network for this regression problem?

4.10 Ollama + Qwen2.5:7B (Local LLM)

WHAT: Ollama is a tool that runs open-weight large language models locally on your own machine and exposes them through a simple API. Qwen2.5:7B is a 7-billion-parameter open LLM from Alibaba's Qwen family.

WHY: Running the LLM locally avoids external API costs and keeps data on-device, at the cost of needing local compute and generally getting a smaller/less capable model than a large hosted API model.

HOW (basic mechanics): The Ollama Python client sends a chat-style request (a list of messages) to the locally running Ollama server, which loads the model and streams/returns generated text back.

WHERE USED IN MY PROJECT: ``services/llm_service.py`` (``LLMService.generate()``) calls ``ollama.chat(model="qwen2.5:7b", messages=[...], options={temperature: 0.1, top_p: 0.9, num_predict: 400, num_ctx: 4096})``.

Interview questions this could prompt:

- Why is temperature set as low as 0.1 for this use case?
- What does ``num_ctx`` (context window size) control, and what happens if the prompt exceeds it?
- What are the trade-offs of a local 7B model versus calling a hosted API model like GPT-4 or Claude?

4.11 React

WHAT: A JavaScript library for building user interfaces out of reusable, composable components, using a virtual DOM to efficiently update the browser DOM when state changes.

WHY: Makes it straightforward to build a dynamic, interactive single-page application (chat UI, live dashboard) without manually manipulating the DOM.

HOW (basic mechanics): Components are JavaScript functions that return JSX (HTML-like syntax); ``useState`` holds local component state, ``useEffect`` runs side effects (like fetching data) in response to state/prop changes; React re-renders a component whenever its state changes.

WHERE USED IN MY PROJECT: The entire frontend (`frontend/src/App.jsx`) - `AuthScreen`, `ResearchDashboard`, chat message list, document upload UI, and more, all as React function components.

Interview questions this could prompt:

- What is the virtual DOM and why does it make updates efficient?
- What is the difference between state and props?
- Why did you choose plain `useState`/`useEffect` instead of a state-management library like Redux?

4.12 Vite

WHAT: A modern frontend build tool and development server, known for very fast startup and hot-module reload during development.

WHY: Faster developer feedback loop than older tools like Create React App/Webpack, and produces optimized, smaller production bundles.

HOW (basic mechanics): During development, Vite serves source files directly over native ES modules and only compiles what's actually requested by the browser (fast cold start); for production, `vite build` bundles and minifies everything.

WHERE USED IN MY PROJECT: `frontend/vite.config.js` configures it; `npm run dev` starts the Vite dev server (the README documents this as the way to run the frontend).

Interview questions this could prompt:

- What is Hot Module Replacement (HMR) and how does Vite achieve fast reloads?
- What is the difference between Vite's dev-server behaviour and its production build output?

4.13 REST APIs / HTTP

WHAT: REST (Representational State Transfer) is an architectural style for designing networked APIs around resources, identified by URLs, manipulated with standard HTTP methods (GET, POST, PATCH, DELETE).

WHY: It is the standard way for a frontend and backend to communicate over the network in a stateless, predictable, cacheable way.

HOW (basic mechanics): A client sends an HTTP request (method + URL + headers + optional JSON body) to a server; the server responds with a status code and, usually, a JSON body.

WHERE USED IN MY PROJECT: Every endpoint in `api/*.py` - e.g. `POST /ask`, `POST /auth/login`, `GET /conversations`, `DELETE /conversations/{id}`, `POST /user/upload`.

Interview questions this could prompt:

- Why is `DELETE /conversations/{id}` a DELETE and not a POST?
- What HTTP status codes does this project return, and where (401, 400, 404, 500)?
- What makes an API 'stateless', and where does this project actually keep state (hint: the database, not the server process)?

4.14 JSON

WHAT: JavaScript Object Notation - a lightweight, text-based, language-independent data format for representing structured data as key-value pairs, arrays, strings, numbers, booleans, and null.

WHY: It is the universal format both the frontend and backend use to exchange data over HTTP, and it maps naturally onto both JavaScript objects and Python dicts.

HOW (basic mechanics): Python's Pydantic models (or plain dicts returned from FastAPI routes) are automatically serialized to JSON text on the way out, and JSON in the request body is automatically parsed back into Python objects on the way in.

WHERE USED IN MY PROJECT: Every request/response body in the entire API; also the JSON Lines (`.jsonl`) log file format used by `services/logging\_service.py` to store one JSON object per retrieval event.

Interview questions this could prompt:

- What's the difference between JSON and JSONL (JSON Lines), and why might JSONL be preferred for logs?
- What data types does JSON support, and how does that map onto Python's types?

Core Computer Science Connections

Format: **Concept** -> **Where it appears** -> **Why it is used** -> **Possible interview question**.

Object-Oriented Programming (OOP)

Where it appears: Nearly every service is a class with a single responsibility: ``AdaptiveHybridRetriever``, ``AdaptiveScoreFusion``, ``RetrievalFeatureService``, ``CrossEncoderReranker``, ``PromptBuilder``, ``LLMService``, ``BM25Retriever``, ``AdaptivePredictorV4``, ``DocumentChunker``, ``DocumentExtractor``, ``UserRAGPipeline``.

Why it is used: Each class bundles related state (e.g. a loaded model, a cache) with the methods that operate on it, and each module then exposes ONE singleton instance (e.g. ``reranker = CrossEncoderReranker()``) that the rest of the app imports and reuses - a lightweight, module-level singleton pattern.

Possible interview question: Why does this project use a class with a singleton instance instead of just plain module-level functions?

Encapsulation

Where it appears: ``BM25Retriever`` hides cache validation, fingerprinting, and index-building behind a simple public ``search()`` method. ``auth_service.py`` hides SQL and password hashing behind functions like ``authenticate_user()``.

Why it is used: Callers (like ``hybrid_service.py`` or ``api/auth.py``) don't need to know HOW the BM25 index is cached or HOW a password is hashed - they just call a clean function.

Possible interview question: What would break if ``hybrid_service.py`` needed to know the internal cache file format of ``BM25Retriever``?

Data Structures - Hash Maps (dict)

Where it appears: ``fusion_service.py``'s ``fuse()`` method uses a Python ``dict`` keyed by a stable document ID to merge BM25 and dense results that refer to the same chunk into a single row.

Why it is used: A hash map gives average $O(1)$ lookup/insert, so merging two lists of up to 50 documents each stays fast instead of doing an $O(n^2)$ nested-loop comparison.

Possible interview question: What is the time complexity of merging the BM25 and dense result lists using a dict versus using nested loops, and why does that matter here?

Data Structures - Sets

Where it appears: ``retrieval_feature_service.py`` converts BM25 and dense document lists to Python ``set``'s to compute ``overlap_ratio`` (a Jaccard-style intersection-over-union).

Why it is used: Set intersection/union in Python is implemented with hash tables internally, so overlap can be computed in roughly $O(n)$ instead of comparing every pair of documents.

Possible interview question: How would you compute the overlap between two ranked lists of 50 documents each without using sets, and what would the complexity be?

Algorithms - Sorting

Where it appears: Every ranked result list (BM25 top-k via ``np.argsort``, the fused results via Python's built-in ``sorted()``, the reranked list) relies on sorting by a score.

Why it is used: Retrieval is fundamentally a ranking problem - the value of the system is in getting the ORDER of results right, so sorting by score is the mechanism that actually produces a ranked list.

Possible interview question: Python's ``sorted()``/``list.sort()`` uses Timsort. What is Timsort's time complexity, and why is it a good fit for this kind of partially-ordered, real-world data?

Algorithms - Search / Information Retrieval

Where it appears: BM25 (probabilistic ranking function based on term frequency and inverse document frequency) and approximate nearest-neighbour vector search (inside ChromaDB) are both retrieval/search algorithms at the heart of the project.

Why it is used: These are the two fundamentally different retrieval paradigms (lexical vs. semantic) that the whole adaptive-fusion idea exists to balance.

Possible interview question: In plain terms, what does BM25 actually compute, and why can two completely different algorithms both be called 'search'?

DBMS - Relational Schema Design

Where it appears: `auth_service.py` defines 4 tables (users, sessions, email_verification_tokens, password_reset_tokens) with `FOREIGN KEY` references to `users(id)`. `conversation_service.py` defines conversations and messages with a foreign key from messages to conversations, `ON DELETE CASCADE`, and two indexes.

Why it is used: Foreign keys enforce referential integrity (a session can't reference a non-existent user); indexes on `user_id` and `conversation_id` make the common lookup queries ("get all conversations for this user") fast instead of scanning the whole table.

Possible interview question: Why index `conversations(user_id)` and `messages(conversation_id)` specifically - what queries does that speed up?

DBMS - CRUD Operations

Where it appears: Full Create/Read/Update/Delete is implemented by hand with raw SQL: `create_conversation` (Create), `list_conversations/get_messages` (Read), `update_title` (Update), `delete_conversation` (Delete, including deleting dependent messages first).

Why it is used: This is the standard pattern for any application with persistent state; understanding it by hand (rather than through an ORM) shows direct SQL literacy.

Possible interview question: Why does `delete_conversation` delete from `messages` before deleting from `conversations`, even though the foreign key has `ON DELETE CASCADE` declared?

DBMS - Parameterized Queries / SQL Injection Prevention

Where it appears: Every SQL statement in `auth_service.py` and `conversation_service.py` uses `?` placeholders with a tuple of parameters (e.g. `connection.execute("SELECT * FROM users WHERE username = ?", (value,))`), never raw string concatenation of user input into SQL.

Why it is used: This is the standard defence against SQL injection - if the query text is fixed and only the data changes, an attacker cannot smuggle SQL syntax into the query through user input.

Possible interview question: What would happen to this app's security if `create_user` built its SQL with an f-string instead of `?` placeholders?

Operating Systems - Concurrency / Threading

Where it appears: `sqlite3.connect(DB_PATH, check_same_thread=False)` explicitly allows a SQLite connection to be used from a different thread than the one that created it, since FastAPI/Uvicorn can serve requests on different worker threads.

Why it is used: Without this flag, SQLite would raise an error if a database connection created on one thread were used from another - a real concurrency concern for a multi-request web server.

Possible interview question: What are the actual risks of `check_same_thread=False` if two requests write to the same SQLite file at the same time?

Operating Systems - File I/O and Atomic Writes

Where it appears: `bm25_service.py`'s `_save_cache()` writes the pickle cache to a temporary file and then calls `os.replace(temporary_path, self.cache_path)` to atomically swap it into place.

Why it is used: `os.replace()` is atomic at the filesystem level on the same volume, so a crash or power loss mid-write cannot leave a half-written, corrupted cache file behind - the old file is only replaced once the new one is fully written.

Possible interview question: Why is writing directly to `self.cache_path` with `open(..., 'wb')` risky compared to the write-then-atomic-rename pattern used here?

Computer Networks - CORS

Where it appears: `app.py` configures `CORSMiddleware` with `allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"]` and `allow_credentials=True`.

Why it is used: Browsers block a webpage from one origin (the Vite dev server on port 5173) from making requests to a different origin (the FastAPI server on port 8000) unless the server explicitly allows it via CORS headers - this configuration is what makes the React app's `fetch()` calls to the backend actually work.

Possible interview question: What is the same-origin policy, and what specific browser behaviour would you see if `allow_origins` did NOT include the frontend's actual origin?

Computer Networks - HTTP Authentication (Bearer Tokens)

Where it appears: `get_current_user()` in `api/auth.py` reads the `Authorization` header, checks it starts with `"Bearer "`, and looks the token up in the sessions table.

Why it is used: The Bearer token scheme is a simple, standard way to prove identity on every request without re-sending a password, and works cleanly with a stateless HTTP request/response model.

Possible interview question: What is the difference between this token-based approach and cookie-based session authentication, and what would change if this were deployed across multiple domains?

Software Engineering - Separation of Concerns / Layered Architecture

Where it appears: The codebase is split into `api/` (HTTP layer), `services/` (business logic), and implicitly a data layer (SQLite/Chroma access inside the relevant service files) - each API route handler is thin and delegates real work to a service function.

Why it is used: This keeps HTTP-specific code (status codes, request parsing) separate from the actual retrieval/ML/auth logic, which makes the logic reusable (e.g. `hybrid_search()` is also called directly from CLI test blocks) and easier to reason about.

Possible interview question: If you wanted to add a GraphQL API alongside the existing REST API, how much of `services/` would you need to change?

Exception Handling

Where it appears: A global exception handler is registered in `app.py` (`api/exceptions.py`); route handlers wrap risky calls in `try/except` and translate errors into proper HTTP status codes (e.g. `HTTPException(status_code=500, detail=str(error))` in the `/ask` route; `HTTPException(status_code=401, ...)` for bad credentials).

Why it is used: Prevents an unhandled Python exception from crashing the server process or leaking a raw stack trace to the client, while still surfacing a meaningful error message.

Possible interview question: What is the difference between an `HTTPException` raised intentionally versus an unexpected Python exception caught by the global exception handler, and how does the client experience differ?

Complexity Analysis

Where it appears: BM25 scoring over the whole corpus is roughly $O(\text{number of documents})$ per query (rank-bm25 is not index-pruned); the fusion step is roughly $O(k \log k)$ for sorting k candidates; sorting

cross-encoder results is likewise $O(k \log k)$ for the small reranking shortlist.

Why it is used: Understanding these costs explains WHY the pipeline retrieves only CANDIDATE_K=50 documents from each retriever (bounding k) instead of reranking or fusing over the entire corpus.

Possible interview question: What is the time complexity of BM25Okapi's ``get_scores()`` call in terms of corpus size, and why does that justify caching the BM25 index instead of rebuilding it per query?

Core Code Logic (Input -> Processing -> Output)

6.1 Feature: Adaptive Hybrid Retrieval

```
INPUT: query = "what are the symptoms of diabetes"
|
PROCESSING:
1. dense_search(query, top_k=50) -> embed query, query ChromaDB
2. bm25_search(query, top_k=50) -> tokenize, BM25Okapi.get_scores()
|
LOGIC:
3. retrieval_feature_service.extract(...) -> 17 numeric features
|
MODEL:
4. adaptive_predictor_v4.predict(features) -> ExtraTreesRegressor
   -> predicted_bm25_weight (clipped to [0, 1])
   -> dense_weight = 1 - bm25_weight
|
LOGIC:
5. fusion_service.fuse(...) -> normalize both signals (min-max),
   weighted sum, merge by stable document key, sort descending
|
OUTPUT: top-10 fused, citation-tagged documents + the weights used
```

Algorithm: the core novel algorithm here is the learned weight prediction itself, followed by a weighted linear combination (a form of late/score fusion, sometimes called Convex Combination or CombSUM with learned weights in IR literature).

Why this approach: compared to a fixed 50/50 weight, this lets the same pipeline behave more like pure BM25 for keyword-heavy queries and more like pure dense search for conceptual queries, without needing two separate pipelines or manual query classification rules.

Time complexity: retrieving 50 candidates from each retriever, extracting 17 features ($O(50)$ work), predicting with 500 trees (each a small, fast traversal), and fusing/sorting 50-100 merged candidates ($O(k \log k)$) - overall this is fast relative to the LLM generation step, which dominates total latency.

Space complexity: the persisted BM25 index and the fused-result dicts are the main memory users; the BM25 cache holds the whole corpus's documents, IDs, metadata, and tokenized text in memory once loaded.

Alternative approaches: (1) Reciprocal Rank Fusion (RRF) - a rank-based fusion that ignores raw scores entirely (the project does have an `rrf\_service.py`, so this was evidently explored/available as an alternative); (2) a fixed-weight hybrid (the project's own baseline for comparison); (3) a learned-to-rank model that scores documents directly instead of predicting a fusion weight.

6.2 Feature: Password Authentication

```
INPUT: username_or_email, password (from POST /auth/login)
|
PROCESSING:
1. SELECT * FROM users WHERE username = ? OR email = ?
2. If no row found -> reject (401)
|
LOGIC:
3. verify_password(password, stored_salt, stored_hash):
   recomputed = PBKDF2-HMAC-SHA256(password, stored_salt, 120000 iterations)
   return hmac.compare_digest(recomputed, stored_hash)
```

```
|
DATABASE:
4. If valid -> UPDATE users SET last_login = now()
5. token = secrets.token_urlsafe(48)
INSERT INTO sessions (token, user_id)
|
OUTPUT: { success: true, user: {...}, token: "<opaque session token>" }
```

Algorithm: PBKDF2 (Password-Based Key Derivation Function 2) - a key-stretching algorithm that deliberately repeats a hash function (SHA-256 here) 120,000 times to make brute-force password guessing computationally expensive, even if the database is stolen.

Step-by-step example: password "mypassword123" + a random 16-byte salt -> run through PBKDF2 with SHA-256, 120,000 rounds -> a fixed-length hash. The salt is stored alongside the hash (in hex) so the same computation can be repeated at login time and compared.

Why PBKDF2 over plain SHA-256: a single SHA-256 hash of a password is extremely fast to compute, which makes brute-forcing millions of password guesses per second feasible on modern hardware; PBKDF2's repeated iterations make each single guess much slower, directly slowing down an attacker.

6.3 Feature: Document Upload -> Chunking -> Indexing (User RAG)

```
INPUT: an uploaded PDF/DOCX/TXT file (POST /user/upload)
|
PROCESSING:
1. Save to a temporary file (tempfile.NamedTemporaryFile)
2. document_extractor.extract(file_path) -> plain text
   - PDF: pypdf, page by page, tagged "[Page N]"
   - DOCX: python-docx, paragraph by paragraph
   - TXT: read directly (UTF-8, errors ignored)
|
LOGIC:
3. document_chunker.chunk(text) -> overlapping 500-char chunks
   (100-char overlap, sliding window)
|
MODEL:
4. embedding_model.encode(chunk) for every chunk -> 384-dim vectors
|
DATABASE:
5. user_document_indexer writes {id, embedding, document, metadata}
   into a Chroma collection scoped to this user_id
|
OUTPUT: { success: true, uploaded_file_name, chunks indexed, ... }
```

Why chunk at all: both the embedding model and the LLM have limited effective input size, and retrieval needs to point at small, specific pieces of a document rather than the whole file, so the document must be split before it can be embedded and searched.

Why overlap the chunks: a hard, non-overlapping split can cut a sentence or an idea exactly across a chunk boundary; a 100-character overlap means the boundary text still appears whole inside at least one chunk.

6.4 Feature: Answering From User Documents (with Context Expansion)

```
INPUT: user_id, query (POST /user/ask or /conversations/{id}/messages)
|
PROCESSING:
1. user_adaptive_retriever.search(user_id, query) -> BM25 + Dense
   + adaptive fusion, SCOPED to that user's Chroma collection only
```

```

|
LOGIC:
2. reranker.rerank(...) -> cross-encoder re-scores, top 5 kept
3. _expand_context(...):
for each top reranked chunk, look up its document_id and
chunk_index, then fetch NEIGHBOURING chunks (+/- window)
from the same document so the LLM gets connected context
instead of isolated fragments
|
LOGIC:
4. prompt_builder.build_prompt(query, expanded_context)
|
MODEL:
5. llm_service.generate(prompt) -> Ollama -> Qwen2.5:7B
|
OUTPUT: grounded answer + weights + documents + per-stage latency

```

6.5 Data Structure Deep Dive: The Fusion Merge (Hash-Map Join)

The most interesting data-structure decision in the project is how BM25 and dense results (two separate lists, each up to 50 items) are merged into one ranked list without duplicating documents that both retrievers found.

Data structure used	A Python `dict`, keyed by `_document_key()` (prefers `chunk_id`, falls back to `document_id`, then the retriever's raw ID, then a hash of the document text as a last resort).
Operations performed	Insert-or-update for every dense result, then insert-or-update for every BM25 result, then a final pass to compute `fusion_score` for every merged entry, then a full sort.
Complexity	$O(n)$ to build the merged dict (n = up to 100 total entries from both retrievers), plus $O(n \log n)$ for the final sort - dominated by the sort.
Why a dict and not two separate lists merged with nested loops	A nested-loop comparison to check "does this BM25 document also appear in the dense list?" would cost $O(n^2)$ in the worst case; a hash map lookup by key is $O(1)$ average, making the whole merge close to linear.
Alternative data structure	A `list` with linear search for each merge, or two indexed pandas DataFrames joined by ID (heavier-weight, more overhead for only ~100 rows per query).

Database Design (SQLite + ChromaDB)

7.1 Why two different databases?

The project uses SQLite for structured, relational data (users, sessions, tokens, conversations, messages) and ChromaDB for high-dimensional vector data (document embeddings). This is a common and sensible split: relational databases are built for exact-match, foreign-key-linked structured records, while vector databases are purpose-built for nearest-neighbour similarity search over embeddings - a query SQLite cannot do efficiently.

7.2 SQLite Database 1: app.db (Authentication)

Table	Key Columns	Purpose
users	id (PK), username (UNIQUE), email (UNIQUE), password_hash, password_salt, email_verified, created_at, last_login	One row per registered user.
sessions	token (PK), user_id (FK -> users.id), created_at	One row per active login session (opaque Bearer token).
email_verification_tokens	token (PK), user_id (FK), expires_at, created_at	Short-lived token e-mailed (in production) to confirm a user's email.
password_reset_tokens	token (PK), user_id (FK), expires_at, used, created_at	Short-lived, single-use token for the forgot-password flow.

Primary keys: `id` on users, `token` on the other three tables (the token itself IS the primary key - it must be unique and unguessable, which a cryptographically random 48-byte URL-safe token satisfies).

Foreign keys: every non-user table has `user\_id REFERENCES users(id)`, enforcing that a session or token can never point at a user that doesn't exist.

Normalization: this schema is in Third Normal Form (3NF) for practical purposes - there is no repeated/redundant user data stored in the sessions or token tables beyond the foreign key itself.

Example queries actually used, explained

Login lookup	`SELECT * FROM users WHERE username = ? OR email = ?` -> WHAT: finds a user by either identifier. WHY: lets a user log in with either field. HOW: a full-table scan unless an index exists on username/email - here, `UNIQUE` constraints on both columns create an implicit index in SQLite, so this lookup is fast even without an explicit `CREATE INDEX`.
Session invalidation on reset	`DELETE FROM sessions WHERE user_id = ?` -> WHAT: removes every active session for a user. WHY: security best practice - after a password reset, old (possibly stolen) session tokens should stop working immediately. HOW: a simple DELETE filtered by the foreign key.

Verify-and-consume token	Read the token row, check `expires_at` against the current time, then `UPDATE users SET email_verified = 1` and `DELETE FROM email_verification_tokens WHERE token = ?` -> WHAT: marks an email verified and destroys the now-used token. WHY: tokens must be single-use and time-limited. HOW: two statements committed together as one logical operation.
---------------------------------	---

7.3 SQLite Database 2: rag_app.db (Conversations)

Table	Key Columns	Purpose
conversations	id (PK), user_id, title, created_at, updated_at	One row per chat conversation/thread a user has started.
messages	id (PK), user_id, conversation_id (FK -> conversations.id, ON DELETE CASCADE), role, content, created_at	One row per individual chat message (user or assistant).

Indexes: `idx\_conversations\_user` on `conversations(user\_id)` and `idx\_messages\_conversation` on `messages(conversation\_id)` - both created explicitly with `CREATE INDEX IF NOT EXISTS`, speeding up the two most common access patterns: "list this user's conversations" and "get all messages in this conversation".

ON DELETE CASCADE: deleting a conversation automatically deletes its messages at the database level. Interestingly, `delete\_conversation()` in the code ALSO explicitly runs `DELETE FROM messages WHERE conversation\_id = ?` before deleting the conversation itself, which is redundant given the CASCADE constraint but defensive (it works even if the CASCADE were ever misconfigured or foreign key enforcement were off - SQLite requires `PRAGMA foreign\_keys = ON` for CASCADE to actually fire, and this project does not appear to enable that pragma anywhere).

HONESTY NOTE: The code path that relies on `ON DELETE CASCADE` in the schema will only actually cascade if SQLite's foreign key enforcement is turned on with `PRAGMA foreign\_keys = ON`, and no such pragma call was found in the code. In practice this doesn't break anything because `delete\_conversation()` deletes messages explicitly anyway - but if asked, be precise about this rather than claiming the cascade is guaranteed to fire.

7.4 ChromaDB (Vector Store)

What it stores	For every chunk: the embedding vector (384 floats for all-MiniLM-L6-v2), the raw chunk text, a unique ID, and a metadata dict (e.g. `source`, `document_id`, `chunk_id`, `chunk_index`, `domain`, `dataset` for the shared corpus; `file_id`, `file_name`, `document_type` for user documents).
Collections	`adaptive_rag_beir_v2` - the large shared, pre-built multi-domain corpus (BEIR-style benchmark data, 75,000+ documents/chunks, referenced by `bm25_service.py`'s collection count check). Separately, one collection per user for the User RAG feature.
Query operation	`collection.query(query_embeddings=[...], n_results=top_k, include=["documents", "distances", "metadatas"])` - a nearest-neighbour similarity search returning the closest vectors, their distances, the original text, and metadata.

"Distance"
"similarity"

vs

ChromaDB returns a distance (lower = more similar); the project converts this to a similarity score with $1 / (1 + \text{distance})$ in `fusion_service.py` so it can be combined with BM25 scores (where higher = more relevant) on a consistent "higher is better" scale.

7.5 Data Flow Through the Databases

User registers -> users (app.db)
User logs in -> sessions (app.db)
User uploads a document -> Chroma collection (per-user, vector store)
User starts chatting -> conversations, messages (rag_app.db)
Every retrieval request -> logs/retrieval_logs.jsonl (flat file, NOT a database - see Part 19 for why this is a JSONL log, not a DB table)

7.6 Possible DBMS / SQL Interview Questions For This Project

- Why did you choose SQLite instead of PostgreSQL or MySQL for this project?
- Walk me through what happens in the database when a user resets their password.
- Why store the salt separately instead of only storing the final password hash?
- What indexes exist in this schema, and what query do each of them speed up?
- How would you migrate this schema if you needed multiple backend server instances writing concurrently?
- What is the difference between `UNIQUE` and `PRIMARY KEY` in the `users` table's design (username/email are UNIQUE, but `id` is the PRIMARY KEY)?
- How does a vector database's 'query' conceptually differ from a SQL `SELECT ... WHERE` - what is it actually computing?

API / Backend Design

8.1 What is an API, in this project's context?

An API (Application Programming Interface) here is the set of HTTP endpoints the FastAPI backend exposes so the React frontend (or any other client, such as a script using `requests` or Postman) can trigger backend behaviour - log in, ask a question, upload a document - without needing to know anything about the Python code, SQLite schema, or ML model underneath.

8.2 Client-Server Communication in This Project

- The browser (client) runs the React app on `http://localhost:5173` (Vite dev server).
- The React app makes HTTP requests via the browser's `fetch()` API to `http://localhost:8000` (the FastAPI/Uvicorn server), configured through `API\_BASE` and the `apiUrl()` helper in App.jsx.
- Every response is JSON; FastAPI handles serialization automatically from Pydantic models or plain dicts.
- Authenticated requests carry `Authorization: Bearer `, read server-side by `get\_current\_user()`.

8.3 Complete Endpoint List

Method	Endpoint	Purpose
GET	/	Root health message.
GET	/health	Simple health check.
GET	/info	Returns project metadata: LLM, embedding model, reranker, fusion strategy, version.
POST	/ask	Ask a question against the SHARED corpus (main RAG pipeline).
GET	/analytics/summary	Aggregated retrieval analytics from the JSONL logs.
GET	/analytics/weights	Distribution of adaptive BM25/dense weights over recent queries.
GET	/analytics/models	Which adaptive model version has been used.
GET	/analytics/recent	The most recent N logged queries.
POST	/auth/register	Create a new user account; also creates a verification token + session.
GET	/auth/verify-email	Consume an email verification token.
POST	/auth/login	Authenticate and issue a session token.
GET	/auth/me	Return the currently authenticated user (requires Bearer token).
POST	/auth/forgot-password	Issue a password reset token for an email (if it exists).

Method	Endpoint	Purpose
POST	/auth/reset-password	Consume a reset token and set a new password; invalidates sessions.
POST	/auth/logout	Delete the current session token.
POST	/user/upload	Upload a PDF/DOCX/TXT file; extract, chunk, embed, index for this user.
GET	/user/documents	List a user's uploaded documents and chunk counts.
GET	/user/{user_id}/documents	List documents for a given user id (used by user_chat.py).
POST	/user/ask	Ask a question scoped to a user's own uploaded documents.
POST	/conversations	Create a new conversation.
GET	/conversations	List a user's conversations.
GET	/conversations/{id}	Get one conversation's details.
POST	/conversations/{id}/messages	Post a user message; runs the RAG pipeline and stores both the user and assistant messages.
PATCH	/conversations/{id}	Rename a conversation's title.
DELETE	/conversations/{id}	Delete a conversation and its messages.

8.4 Deep Dive: POST /ask

Method / Endpoint	POST /ask
Purpose	The main entry point for asking a question against the pre-built shared corpus.
Input	JSON body `{ "query": "..."} `, validated by the `QueryRequest` Pydantic model.
Output	JSON `{ success, processing_time, query, answer, weights, documents, latency, model } `, shaped by the `QueryResponse` Pydantic model (with `extra = "allow"` so additional fields don't break validation).
Where it is used	Called from the React chat UI for the main/shared knowledge base chat mode.
What happens internally	The handler times the request, calls `rag_pipeline.answer(query=...)`, which internally runs hybrid retrieval -> reranking -> prompt building -> LLM generation (see Part 6.1 and the pipeline diagram in Part 1.7), and returns the combined result. Any exception is caught and turned into an `HTTPException(status_code=500, detail=str(error))` rather than crashing the server.

8.5 Deep Dive: POST /auth/register

Method / Endpoint	POST /auth/register
Purpose	Create a new user account and immediately log them in.

Input	<code>{ username, email, password }</code> validated with length constraints (<code>Field(min_length=3, max_length=30)</code> for username, <code>min_length=6</code> for password).
Output	<code>{ success, message, user, token, verification_token }</code> . The verification token is returned directly in the response body, explicitly commented in the code as a development-mode shortcut in place of actually sending an email.
Validation	Pydantic validates types/lengths automatically; <code>create_user()</code> additionally checks for a duplicate username/email and raises a <code>ValueError</code> , which the route converts into HTTP 400.
What happens internally	A new user row is inserted with a freshly hashed password, a verification token row is inserted, and a session token row is inserted (so the new user is logged in immediately without a separate login step).

8.6 HTTP Methods Used, and Why

Method	Used for in this project	Why this method
GET	Reading data: health checks, analytics, listing conversations/documents, verifying email (via query string token).	GET requests are for retrieving data without side effects (aside from email verification, which is a common real-world exception where a GET link is clicked from an email).
POST	Creating things or triggering an action with side effects: login, register, ask a question, upload a file, create a conversation/message.	POST is used when the request creates a resource or has a meaningful side effect and is not naturally idempotent.
PATCH	Renaming a conversation's title.	PATCH is for a partial update to an existing resource (only the title changes, not the whole conversation).
DELETE	Deleting a conversation.	DELETE maps directly and unambiguously to a deletion action, consistent with REST conventions.

8.7 Status Codes Actually Used

- 200 OK - default success response for GET/POST that FastAPI returns automatically.
- 400 Bad Request - invalid registration data, duplicate username/email, invalid/expired verification or reset token, unsupported uploaded file type.
- 401 Unauthorized - missing/invalid Authorization header, invalid session token, wrong login credentials.
- 404 Not Found - used in `list_documents` when the underlying Chroma collection lookup fails.
- 422 Unprocessable Entity - FastAPI's automatic response when a request body fails Pydantic validation (e.g. a password under 6 characters).
- 500 Internal Server Error - used explicitly in the `/ask` and `/user/upload` routes to wrap unexpected exceptions, plus the global exception handler as a last-resort catch-all.

8.8 Authentication on the API

`get_current_user()` (in `api/auth.py`) is a FastAPI dependency (`Depends(get_current_user)`) used on protected routes such as `/auth/me` and `/user/upload`. It reads the `Authorization` header, validates the `Bearer` prefix, and resolves the token to a user via `get_user_from_token()`, which performs a SQL JOIN between `sessions` and `users`. If the token is missing, malformed, or not found in the database, it raises `HTTPException(401)` before the route handler's own code ever runs.

8.9 Input Validation

Two layers: (1) Pydantic model field constraints (`Field(min_length=..., max_length=...)`) catch obviously malformed input automatically and return HTTP 422 with a detailed error message before the route function body even executes; (2) business-rule validation inside the service functions themselves (e.g. `create_user()` re-checks password length and checks for duplicate username/email, which Pydantic alone cannot know about since it requires a database lookup).

8.10 Error Handling Pattern

The consistent pattern across the API is: catch expected error conditions (`ValueError` for business-rule violations) and translate them into a specific `HTTPException` with an appropriate status code and message; catch anything unexpected in a broader `except Exception` and return 500 with the exception's string message; and have a final global exception handler registered on the FastAPI app itself as a safety net for anything that slips past route-level handling.

Frontend Design

9.1 Structure

The frontend is a single-page application (SPA) built with React and Vite. Unlike many React projects, it is NOT split into a `components/` folder of many files - the whole UI lives in one file, `frontend/src/App.jsx` (2,318 lines), styled by one large hand-written stylesheet, `App.css` (3,179 lines). `main.jsx` is the standard Vite/React bootstrap file that mounts `App` into the page's root DOM element.

9.2 Components (all defined inside App.jsx)

- `AuthScreen` - handles both login and registration, toggled by a `mode` state ('login' / 'register').
- `MetricCard`, `PipelineStep`, `MeasurementRow`, `CandidateBox`, `ModelLine`, `WeightBar`, `ContextStat` - small presentational components used to build the Research Dashboard's visual pipeline and metrics display.
- `ResearchDashboard` - the panel showing the live adaptive BM25/dense weights, pipeline stages, latency breakdown, and retrieval configuration for the most recent answer.
- `App` - the top-level component: holds authentication state, the conversation list, the active conversation's messages, the chat input, and orchestrates all the API calls.

9.3 Routing

HONESTY NOTE: No client-side routing library (such as React Router) was found in package.json or imports. The app appears to be a single view that conditionally renders either the `AuthScreen` or the main chat interface based on whether `user` state is set, rather than using URL-based routes for different pages.

9.4 State Management

All state is local component state via React's built-in `useState`/`useEffect`/`useMemo`/`useRef` hooks - no Redux, Zustand, or Context API global store was found. Key state in the top-level `App` component includes: `user` (initialized from `getStoredUser()`, i.e. `localStorage`), `booting`, the conversation list, `documents`, the active conversation id, `messages`, the current `query` text, and several loading flags (`sending`, etc.).

9.5 Events and Forms

Forms are controlled components: each `input`'s value is driven by a `useState` variable, and typing fires an `onChange` handler that updates that state. Submitting a form (login/register, or sending a chat message) is handled by an `onSubmit`/`onClick` handler that calls an `async` function, which in turn calls `apiRequest()` to talk to the backend and updates state with the result (or an error message) when the request resolves.

9.6 API Calls From the Frontend

`apiRequest(path, options)` is the single shared helper for every backend call. It: reads the stored Bearer token, merges it into the request headers, defaults `Content-Type: application/json` unless the body is a `FormData` object (used for file uploads, where the browser must set its own multipart boundary header),

calls `fetch()`, attempts to parse the JSON response, and throws a plain JavaScript `Error` with a meaningful message if the HTTP response was not OK - so every calling component can use one consistent `try { ... } catch (error) { setError(error.message) }` pattern.

9.7 User Journey

1. Page loads -> App checks `localStorage` for a saved user/token
2. If not logged in -> `AuthScreen` (login or register form)
3. On successful login/register -> token + user saved to `localStorage`, App switches to the main chat view
4. App fetches: conversation list, uploaded documents
5. User can: start a new conversation, pick an existing one, upload a document (multipart `FormData` -> `/user/upload`), or type a question and send it
6. Sending a question -> POST to `/ask` or the conversation's `/messages` endpoint -> response rendered as a chat bubble, plus the `ResearchDashboard` updates with the new adaptive weights, pipeline steps, and latency
7. User can log out -> token/user cleared from `localStorage`, `AuthScreen` shown again

9.8 Validation on the Frontend

Basic client-side checks exist (e.g. requiring non-empty fields before enabling a submit action) but the authoritative validation is on the backend (Pydantic field constraints and service-layer checks) - the frontend validation is a UX convenience, not a security boundary, which is the correct general principle: never trust client-side validation alone.

Machine Learning / AI / LLM / RAG Deep Dive

10.1 This project genuinely contains ML, RAG, and LLM components

Confirmed directly from the code: an ML regression model (ExtraTreesRegressor) trained with scikit-learn, a full Retrieval-Augmented Generation pipeline (BM25 + dense retrieval + fusion + reranking + grounded LLM generation), and a locally-served LLM (Qwen2.5:7B via Ollama). This section explains each in depth.

10.2 The Adaptive Weight Model - Dataset

The model's training data is NOT hand-labeled by a human. It is generated automatically by `training/weight_optimizer.py` (PerformanceBasedWeightOptimizer): for every query in an evaluation dataset, the optimizer runs BM25 and dense retrieval once, then tries many BM25/Dense weight combinations (stepping from 0.0 to 1.0 in increments of 0.05), fuses the results with each combination, scores each resulting ranking with real IR metrics, and keeps whichever weight produced the BEST retrieval quality for that specific query. That best weight becomes the training label - `oracle_bm25_weight` - for that query's row in the training dataset.

IMPORTANT: This is an important nuance for an interview: the labels are not "the correct human answer", they are "the empirically best-performing weight found by brute-force search, treated as an oracle/ground truth for supervised training." This is a legitimate and common technique for training a fusion-weight predictor when no human-labeled weights exist.

10.3 Features (input to the model)

Same 17 features described in Part 3.5 / Part 6.1. Summarized as WHAT each measures:

Feature		What it measures
bm25_top_score bm25_mean_score bm25_score_std	/	How high, and how consistent, BM25's confidence is across its top results.
bm25_score_gap bm25_top_mean_ratio	/	How much the single best BM25 result stands out from the rest - a big gap suggests one clearly-best keyword match exists.
dense_top_similarity dense_mean_similarity dense_similarity_std	/	The dense/semantic equivalent of the BM25 confidence stats above.
dense_similarity_gap dense_top_mean_ratio	/	How much the top dense result stands out semantically.
overlap_ratio		What fraction of the BM25 and dense candidate sets agree (Jaccard similarity) - high overlap suggests both retrievers already agree, so the fusion weight matters less.
rank_agreement		Whether documents found by both retrievers appear in similar RELATIVE positions, not just whether they overlap.

Feature	What it measures
query_token_count / query_unique_token_count / query_avg_token_length	Simple query-shape statistics - short queries with repeated or very short tokens behave differently from long, specific queries.
bm25_top_lexical_coverage / dense_top_lexical_coverage	What fraction of the query's own words literally appear in each retriever's top result - the single strongest feature per the README's own reported analysis (dense_top_lexical_coverage).

10.4 Model

Algorithm	ExtraTreesRegressor (Extremely Randomized Trees), from scikit-learn.
Hyperparameters	n_estimators=500, min_samples_leaf=2, max_features=0.8, random_state=42, n_jobs=-1 (uses all CPU cores).
Target	A single continuous number between 0 and 1: the predicted BM25 fusion weight (dense weight is always derived as 1 minus this).
Compared against	A RandomForestRegressor with the same hyperparameters, trained and validated in the same script for a fair comparison (train_v4_rich_model.py).
Train/Val/Test split	70% train / 15% validation / 15% test, via scikit-learn's train_test_split with random_state=42 (reproducible split).
Evaluation metrics on the regression itself	MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), and R ² (coefficient of determination) - all computed with scikit-learn's metrics functions.
Persistence	The trained model is serialized with joblib to `models/adaptive_weight_model_v4_rich.pkl` and loaded once at server startup.

HONESTY NOTE: The uploaded ZIP does not include the trained .pkl model file, the ChromaDB folder, or the BM25 cache - these are runtime/build artifacts explicitly excluded from the repository (per the README and .gitignore). This means the code as uploaded cannot actually answer a query end-to-end without first running the corpus-building, model-training, and indexing scripts. Say this plainly if asked whether you can run a live demo right now from just this ZIP.

10.5 Prediction (inference time)

At query time, `AdaptivePredictorV4.predict()` builds a single-row pandas DataFrame with the 17 features in the SAME fixed column order the model was trained on (this ordering matters - scikit-learn tree models take a plain feature matrix and have no concept of column names once trained, only column position), calls `self.model.predict(x)`, and clips the output to `[0, 1]` with `np.clip` as a safety net in case the model ever extrapolates slightly outside that range.

10.6 Complete Query -> Answer Flow (RAG)

Already diagrammed in Part 1.7 and Part 6.1. Restated as the five standard RAG stages:

Stage	What happens	Key file
1. Retrieval	BM25 + dense retrieval run in parallel, 50 candidates each.	bm25_service.py, retrieval_service.py
2. Fusion	17 features extracted -> adaptive weight predicted -> scores normalized and combined.	retrieval_feature_service.py, adaptive_predictor_v4.py, fusion_service.py
3. Reranking	A cross-encoder re-scores the fused shortlist for a more accurate final order.	reranker_service.py
4. Context assembly	Top chunks (with neighbour expansion for user documents) formatted into a strict grounded-QA prompt.	prompt_builder.py, user_pipeline.py
5. Generation	The prompt is sent to a local Qwen2.5:7B model via Ollama; the text response is returned.	llm_service.py

10.7 Chunking (recap from Part 3.8 / Part 6.3)

Fixed-size sliding-window chunking: 500 characters per chunk, 100-character overlap. Simple, predictable, and language-agnostic, but not "semantic" chunking (it does not try to respect sentence or section boundaries) - a reasonable trade-off for a project prioritizing a complete working pipeline over chunking sophistication.

10.8 Embeddings

Embeddings are produced by all-MiniLM-L6-v2 (via Sentence-Transformers), producing a 384-dimensional dense vector per chunk. The SAME model is used to embed both the corpus/documents at indexing time and the incoming query at search time - this consistency is essential, since comparing vectors from two different embedding models would be meaningless.

10.9 Retrieval (BM25 + Dense) and Hybrid/Adaptive Fusion

Covered in full detail in Parts 3, 4, and 6. In one sentence: BM25 gives lexical/keyword matches, dense retrieval gives semantic/paraphrase matches, and instead of a fixed blend, a trained regression model predicts the right blend per query, and the two normalized score signals are combined with that weight.

10.10 Reranking

A cross-encoder (ms-marco-MiniLM-L-6-v2) re-scores the top fused candidates by jointly processing (query, document) pairs, which is more accurate than the bi-encoder similarity used for the initial retrieval, but too slow to run over the whole corpus - hence it only reranks a small shortlist (covered in Part 4.8).

10.11 Context / Prompting

PromptBuilder assembles the final prompt from a fixed system prompt (grounded-QA instructions - see the exact rules quoted in Part 6/Part 4) plus the retrieved document chunks, each clearly delimited as --- Document Chunk N --- ... --- End Chunk N ---, followed by the user's question. The system prompt explicitly instructs the model to answer ONLY from the provided context, not to use outside knowledge, and not to mention the retrieval system itself (BM25, embeddings, reranking) in its answer.

10.12 Generation

Qwen2.5:7B, served locally through Ollama, is called with a low temperature (0.1) for more deterministic, grounded output, top_p=0.9, a generation cap of 400 tokens (num_predict), and a 4,096-token context window (num_ctx). The response's message content is returned as the final answer string.

10.13 Hallucination - How the Project Tries to Reduce It

- The system prompt is explicitly restrictive: "Your ONLY factual source is the DOCUMENT CONTEXT provided below" and "Do NOT add outside knowledge", with an instruction to say so if the context is insufficient rather than inventing an answer.
- Low temperature (0.1) reduces random/creative deviation from the source text.
- Reranking improves the quality/relevance of what actually reaches the prompt, which reduces the chance the model is forced to answer from irrelevant context.
- Context expansion (user RAG) reduces the chance of an incomplete, out-of-context fragment misleading the model.

HONESTY NOTE: There is no automated hallucination-detection or fact-checking step in the code (e.g. no answer-vs-context entailment check). Hallucination reduction here is entirely through prompt design and retrieval quality, not a separate verification stage - be honest about this distinction if asked.

10.14 Evaluation / Metrics

evaluation/metrics.py implements Precision@K, Recall@K, Reciprocal Rank, DCG@K, and nDCG@K by hand (not imported from a library). The main benchmark script, evaluation/benchmark_v4_rich_actual.py (1,333 lines), runs these metrics over a 1,000-query, 4-domain (Argumentation, Finance, Medical, Scientific) evaluation set and compares four retrieval configurations: BM25-only, Dense-only, Fixed Hybrid (fixed 50/50 weight), and Adaptive Hybrid.

Metric	Fixed Hybrid	Adaptive Hybrid	Relative Change
Precision@10	0.1111	0.1148	+3.33%
Recall@10	0.5189	0.5381	+3.70%
MRR	0.4358	0.4340	-0.40%
nDCG@10	0.4073	0.4159	+2.09%

(These figures are reproduced directly from the project's own README, not invented.)

The README is explicit and honest that MRR is slightly LOWER under the adaptive method, and states: "The results show improvements in Precision@10, Recall@10, and nDCG@10 over the fixed hybrid configuration. MRR is slightly lower, so the result should be reported honestly rather than presenting the adaptive method as universally better." This same honesty should carry directly into how you present the project in an interview.

10.15 Limitations (stated by the project itself, or clearly implied by the code)

- MRR does not improve under the adaptive method in the 1,000-query benchmark (explicitly acknowledged).
- Improvements are not universal at the query level: out of 1,000 queries, MRR improved for only 138 and actually decreased for 94 (per the README's own per-query breakdown).

- Retrieval behaviour and benefit vary substantially by domain (e.g. the README's domain table shows very different Precision/Recall/MRR/nDCG numbers across Argumentation, Finance, Medical, and Scientific).
- The chunking strategy is fixed-size character chunking, not sentence- or semantically-aware chunking.
- No hallucination-detection or answer-verification step exists beyond prompt instructions.
- The trained model artifacts, vector index, and BM25 cache are not included in the repository, so results are not reproducible from the ZIP alone without retraining/rebuilding them first.

System Architecture

11.1 High-Level Architecture

```
+-----+
| React Frontend |
| (Vite, port 5173)|
+-----+
| HTTP/JSON (fetch, Bearer token)
v
+-----+
| FastAPI Backend |
| (Uvicorn, :8000) |
+-----+
| | |
+-----+ | | +-----+
| | | |
v v v v
+-----+ +-----+ +-----+
| SQLite (auth) | | SQLite | | ChromaDB |
| app.db | | (chat) | | (vector store: |
+-----+ | rag_app.db| | shared corpus + |
+-----+ | per-user docs) |
+-----+
|
v
+-----+
| BM25 index |
| (rank-bm25, cached |
| to disk / pickle) |
+-----+
|
v (retrieved + fused + reranked context)
+-----+
| Ollama (local server) |
| running Qwen2.5:7B |
+-----+
```

11.2 Retrieval Pipeline Architecture (repeated from Part 1, for completeness)

```
User Query
|
v
+-----+
| Query Processing |
+-----+
|
+-----+ +-----+
| |
v v
+-----+ +-----+
| BM25 Retrieval | | Dense Retrieval|
| Top-50 | | Top-50 |
+-----+ +-----+
| |
+-----+
|
v
```

```

Retrieval Feature Extraction (17 features)
|
v
+-----+
| Adaptive Weight Predictor |
| Extra Trees Regressor   |
+-----+
|
BM25 / Dense weights
|
v
Adaptive Score Fusion
|
v
Top-10 Results
|
v
Cross-Encoder Reranking (Top-5)
|
v
Context Selection / Expansion
|
v
Grounded Prompt Construction
|
v
Qwen2.5:7B (via Ollama)
|
v
Final Answer

```

11.3 Component Responsibilities

Frontend (React)	Presentation and interaction only - it holds no business logic beyond simple UI state; every real decision (retrieval, fusion, generation, auth) happens server-side.
API layer (api/)	Thin HTTP adapters - parse/validate requests, call the right service, shape the HTTP response and status code.
Service layer (services/)	All business logic: retrieval, fusion, ML prediction, reranking, prompting, LLM calls, authentication, conversation persistence.
SQLite	Durable structured storage for accounts and chat history.
ChromaDB	Durable vector storage and nearest-neighbour search.
BM25 cache (pickle file)	A performance optimization - avoids rebuilding a 75K+ document lexical index on every server start.
Ollama + Qwen2.5:7B	A separate local process/server responsible only for text generation, decoupled from the Python backend.

11.4 Is This a Monolith or Microservices?

This is a **monolith**: one Python process (the Uvicorn-served FastAPI app) contains the API layer, all retrieval logic, the ML model, and database access code. Ollama runs as a separate local process, but it is used more like a local dependency/service than a networked microservice in a distributed system sense.

This is a completely reasonable choice for a research/portfolio project at this scale - microservices would add operational complexity without a corresponding benefit here.

How To Run The Project

This is a beginner-friendly walkthrough based directly on the README's own instructions and the actual code's requirements. Commands are reproduced as given in the project.

1. Prerequisites

- Python 3 with pip.
- Node.js with npm (for the Vite/React frontend).
- Git (to clone the repository).
- Ollama installed locally, with the qwen2.5:7b model pulled (required by services/llm_service.py - not explicitly documented in the README's install steps, but required for the /ask and /user/ask endpoints to actually generate an answer).

2. Clone the repository

```
git clone https://github.com/Parasfunde21/adaptive-hybrid-rag.git
cd adaptive-hybrid-rag
```

3. Create and activate a Python virtual environment (Windows PowerShell, as documented)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

4. Install backend dependencies

```
pip install -r backend/requirements.txt
```

HONESTY NOTE: Use backend/requirements.txt, not the root requirements.txt - the root file is UTF-16 encoded and contains many packages (langchain, langgraph, kubernetes, etc.) that do not appear to be imported anywhere in the actual backend code. See Part 19 for the full explanation.

5. Install frontend dependencies

```
cd frontend
npm install
```

6. Database initialization

SQLite tables are created automatically the first time the relevant service module is imported - `auth\_service.py` calls `initialize\_database()` at import time, and `conversation\_service.py` calls `initialize\_database()` at import time. The README additionally documents a manual migration utility:

```
cd backend
python services/migrate_database.py
```

7. Build the vector index and train the adaptive model (required before the app can actually answer queries)

HONESTY NOTE: The README's 'Running the Application' section assumes the ChromaDB collection, the BM25 cache, and the trained adaptive_weight_model_v4_rich.pkl already exist. None of these are included in the ZIP (they are excluded from Git as build/runtime artifacts). To actually run this end-to-end from a fresh clone, you would first need to: build/populate the ChromaDB collection with the corpus, generate training data with the scripts in backend/training/, train the V4 Rich model, and only then start the servers. If asked to 'just run it' in an interview setting, be upfront about this dependency chain rather than claiming a single command boots a fully working demo.

8. Start the backend

```
.\venv\Scripts\Activate.ps1
cd backend
python app.py
```

HONESTY NOTE: Running `python app.py` directly only works if `app.py` itself calls `uvicorn.run(...)` in an `if __name__ == "__main__":` block - based on the file's actual contents (Part 3.1), `app.py` does NOT contain such a block; it only defines the `app` object. The conventional way to actually start this server is `uvicorn app:app --reload` from the backend directory (or `python -m uvicorn app:app`), consistent with the port 8000 the README and frontend both assume. If asked how you actually start the server, describe the uvicorn command rather than the README's literal `python app.py` line.

9. Start the frontend

```
cd frontend
npm run dev
```

Vite will print the local dev server URL (typically `http://localhost:5173`) in the terminal.

10. How to test it

- Manually: open the printed Vite URL, register an account, log in, ask a question or upload a document.
- Via the CLI test blocks built into several service files: e.g. running `python -m services.hybrid_service` or `python -m services.rag_pipeline` directly starts an interactive command-line loop that lets you type queries and inspect retrieval/answer output without the frontend at all - useful for debugging the pipeline in isolation.
- Via the evaluation benchmark: `python -m evaluation.benchmark_v4_rich_actual` (documented in the README) runs the full 1,000-query comparison and writes result CSVs/JSON to the evaluation results directory.

11. Common errors and solutions

Error	Likely cause	Fix
<code>FileNotFoundError: V4 model not found</code>	The trained .pkl model doesn't exist yet.	Run the training scripts in <code>backend/training/</code> to produce <code>models/adaptive_weight_model_v4_rich.pkl</code> .
<code>ValueError: ChromaDB collection ... is empty</code>	The vector database hasn't been populated.	Run the corpus-indexing scripts before starting the server.
CORS error in the browser console	Frontend is running on a different port than 5173, or backend CORS origins weren't updated.	Either run Vite on its default port, or add the actual frontend origin to the <code>allow_origins</code> list in <code>app.py</code> .
Connection refused calling the LLM	Ollama isn't running, or the <code>qwen2.5:7b</code> model wasn't pulled.	Start Ollama and run <code>ollama pull qwen2.5:7b</code> .
401 Unauthorized on protected routes	Missing or expired Bearer token.	Log in again to get a fresh session token.

12. Deployment instructions

HONESTY NOTE: No deployment instructions or configuration were found (no Docker, no cloud config). This project is documented and built for local development only.

Realistic Project Challenges

Split clearly into what the project already handles, and honest, realistic potential weaknesses/improvements - not challenges you personally experienced unless you actually did.

13.1 Implemented in the project

- Avoiding a slow BM25 index rebuild on every restart, solved with a fingerprinted, atomically-written pickle cache (bm25_service.py).
- Comparing incompatible score scales (BM25 raw scores vs. Chroma distances) before combining them, solved with independent min-max normalization and a distance-to-similarity conversion (fusion_service.py).
- Merging two retrieval result sets without duplicating documents found by both, solved with a stable document-key hash-map join (fusion_service.py).
- Preventing the LLM from answering with outside knowledge, addressed with an explicit, restrictive grounded-QA system prompt and a low generation temperature (prompt_builder.py, llm_service.py).
- Preventing isolated, disconnected chunks from confusing the LLM in the user-document flow, addressed with chunk-neighbour context expansion (user_pipeline.py).
- Generating training labels for the adaptive model without manual labeling, addressed with the oracle grid-search weight optimizer (weight_optimizer.py).
- Preventing SQL injection, addressed by using parameterized queries everywhere.
- Preventing stolen session tokens from remaining valid after a password reset, addressed by deleting all sessions for that user on reset.

13.2 Potential improvements / open challenges

- **Reproducibility:** model artifacts, the vector index, and the BM25 cache are excluded from the repo, so a fresh clone cannot immediately answer queries - a documented setup/build script would make the project easier to reproduce.
- **Chunking quality:** fixed 500-character chunking can still split a sentence or table awkwardly despite the 100-character overlap; sentence- or paragraph-aware chunking could improve retrieval precision further.
- **MRR regression:** the adaptive method's own benchmark shows a very small MRR decrease versus the fixed baseline - understanding WHICH query types cause that regression (the README's per-query breakdown shows 94 out of 1,000 queries got worse MRR) is an open investigation, and the project includes analysis scripts (e.g. group_v3_failures.py, error_analysis_v3.py) evidently aimed at exactly this kind of failure analysis.
- **No automated tests:** there is no pytest-based regression test suite guarding against a future code change silently breaking retrieval or auth logic.
- **Single-process scalability:** SQLite and an in-process BM25 index work well for one server instance, but would need real changes (e.g. a networked database, a shared search service) to scale to multiple backend instances behind a load balancer.
- **Local LLM capability ceiling:** Qwen2.5:7B is a relatively small model; answer quality on complex, multi-part questions is inherently more limited than a larger hosted model.
- **Frontend structure:** a single 2,318-line App.jsx file would benefit from being split into separate component files as the UI grows.

- **No hallucination-detection layer:** grounding is currently enforced only through prompting, with no automated check that the final answer is actually consistent with the retrieved context.

13.3 What I would need to do to improve them

This is the section you should personalize honestly based on what you actually did versus what you understand but did not implement. A safe, honest template answer: "I understand these as the next engineering steps - for example, I would add a small script that automates corpus indexing and model training from a single command so the project is reproducible from a fresh clone, and I would look more closely at the 94 queries where the adaptive method actually regressed MRR to understand whether there's a query pattern the current 17 features don't capture well."

Interview Questions By Category

Every question below is grounded in the actual project. Each has: Answer, Technical Explanation, Why asked, and a Follow-up.

14.1 Project Understanding

Q1. In one sentence, what problem does your project solve?

Answer: It replaces a fixed BM25/dense retrieval weight in a hybrid RAG system with a per-query weight predicted by a trained machine learning model.

Technical Explanation: The predictor is an ExtraTreesRegressor trained on 17 features describing retrieval confidence and query shape, with training labels generated by an oracle grid-search over possible weights.

Why an interviewer may ask this: This is the standard opening question - it tests whether you can summarize the project's core contribution without getting lost in implementation detail.

Possible follow-up: Why is a fixed weight actually a problem in practice?

Follow-up answer: Because different queries genuinely favour different retrieval signals - keyword-heavy queries do better with BM25, paraphrased/conceptual queries do better with dense retrieval - so one fixed ratio is a compromise that is never ideal for every query.

Q2. Who is the intended user of this system?

Answer: As built, it supports two flows: a person chatting against a large pre-built shared corpus, and a person uploading and chatting with their own documents.

Why an interviewer may ask this: Tests whether you inflate the scope of the project beyond what the code actually supports.

What I should remember: Be honest that this is a research/portfolio project, not a deployed product with real usage data.

14.2 Architecture

Q3. Describe the system architecture at a high level.

Answer: A React/Vite frontend calls a FastAPI backend over HTTP/JSON with Bearer-token auth. The backend uses two SQLite databases (auth, conversations), ChromaDB for vector storage, a persisted BM25 index, and a local Ollama server running Qwen2.5:7B for generation.

Why an interviewer may ask this: Standard architecture walkthrough question - answer top-down, not file-by-file.

Possible follow-up: Is this a monolith or microservices?

Follow-up answer: A monolith - one Python process contains the API layer, retrieval logic, and ML model. Ollama is a separate local process but is used as a local dependency, not a networked microservice.

Q4. Why is business logic separated into a services/ layer instead of writing it directly in the API route handlers?

Answer: It keeps HTTP-specific concerns (status codes, request parsing) separate from the actual retrieval/ML/auth logic, and makes that logic reusable and testable independently of the web framework.

Technical Explanation: Several service files (e.g. hybrid_service.py, reranker_service.py) even include their own `if __name__ == "__main__":` CLI test blocks, showing the logic can run and be debugged completely outside of FastAPI.

14.3 Programming

Q5. Walk me through what `_document_key()` does in `fusion_service.py` and why it's needed.

Answer: It generates one stable identifier for a chunk so results from BM25 and dense retrieval that refer to the same chunk can be merged into a single row instead of appearing twice.

Technical Explanation: It tries, in priority order: `chunk_id` from metadata, then `document_id`, then the retriever's raw ID, then a fallback hash of the document text itself.

Possible follow-up: Why fall back to hashing the document text as a last resort?

Follow-up answer: Because if none of the more reliable IDs are present in the metadata, some join key is still needed to detect duplicates - a text hash is a reasonable last resort, though it would fail to merge two chunks with identical text but genuinely different provenance (a known limitation of that fallback).

Q6. Why does `retrieval_feature_service.py`'s `extract()` method support four different calling conventions?

Answer: To stay backward compatible with older (V3) call sites while supporting the new (V4) call sites, without having to update every place in the code that calls it.

Why an interviewer may ask this: Tests whether you understand defensive API design versus just reading it as unnecessary complexity.

Possible follow-up: What would you do differently if you were designing this from scratch today?

Follow-up answer: I would likely pick one consistent keyword-argument signature and update all call sites, since the multiple-signature approach adds real complexity - the backward-compatibility shim made sense during iterative development but isn't something I'd keep long-term.

14.4 OOP

Q7. Give an example of encapsulation in your project.

Answer: BM25Retriever hides all of its cache validation, fingerprinting, and index-building logic behind one public `search()` method.

Possible follow-up: What's the benefit of that, concretely, in this codebase?

Follow-up answer: `hybrid_service.py` can call `bm25_search(query, top_k)` without knowing or caring whether the index came from a cache or was freshly built - the internal implementation could change completely without breaking any caller.

14.5 Data Structures & Algorithms

Q8. What data structure is used to merge the BM25 and dense result lists, and why?

Answer: A Python dict keyed by a stable document key.

Technical Explanation: Average $O(1)$ lookup/insert per document versus $O(n)$ per document with a linear/list search, so merging up to 100 total candidates stays close to linear overall instead of quadratic.

Possible follow-up: What is the overall time complexity of the fusion step?

Follow-up answer: Building the merged dict is $O(n)$; the final sort by fusion score is $O(n \log n)$ - the sort dominates, so the whole fusion step is $O(n \log n)$ where n is the number of unique candidates (at most 100).

Q9. Explain BM25 conceptually, without the formula.

Answer: BM25 scores a document higher when it contains the query's words more often, adjusted so that very long documents aren't unfairly favoured just for being long, and so that a term appearing many times gives diminishing extra credit rather than unlimited extra credit.

Technical Explanation: Formally BM25 combines term frequency (with saturation), inverse document frequency (rarer terms count for more), and document-length normalization.

Possible follow-up: Why does term frequency 'saturate' instead of scaling linearly?

Follow-up answer: Without saturation, a document that repeats a query word 100 times would dominate purely by repetition rather than genuine relevance; saturation means additional occurrences past a point add only a small amount of extra score.

14.6 DBMS / SQL

Q10. Why use raw SQL with sqlite3 instead of an ORM like SQLAlchemy?

Answer: The project uses Python's built-in sqlite3 module directly, with parameterized `?` placeholder queries, rather than an ORM.

Technical Explanation: Raw SQL gives full control and no extra dependency, at the cost of more boilerplate for schema/migration management compared to an ORM like SQLAlchemy (which does appear in the root requirements.txt, but is not actually imported anywhere in the backend service code).

Why an interviewer may ask this: Tests whether you understand the trade-off, not just that you avoided a library.

Possible follow-up: What would change if you added SQLAlchemy?

Follow-up answer: You'd get schema definitions as Python classes, automatic parameter binding, and easier migrations, at the cost of an additional abstraction layer over the SQL that's currently written and understood explicitly.

14.7 Backend

Q11. How does your app protect an endpoint like /auth/me from unauthenticated access?

Answer: Via a FastAPI dependency, `get\_current\_user`, injected with `Depends(get\_current\_user)`.

Technical Explanation: It reads the Authorization header, requires a 'Bearer ' prefix, extracts the token, and looks it up against the sessions table joined with users; a missing/invalid/unknown token raises HTTPException(401) before the route's own logic runs.

14.8 Frontend

Q12. How does the frontend know if a user is logged in after a page refresh?

Answer: On load, `getStoredUser()` and `getStoredToken()` read the previously saved user object and Bearer token from localStorage, so React state is initialized as already-logged-in instead of starting from scratch.

Possible follow-up: What's a downside of storing the auth token in localStorage?

Follow-up answer: Unlike an HttpOnly cookie, localStorage is readable by any JavaScript running on the page, so it is more exposed to XSS (cross-site scripting) attacks - a real trade-off to be aware of even though it's simpler to implement.

14.9 APIs

Q13. Why is `/ask` a POST endpoint and not a GET with a query string parameter?

Answer: Asking a question triggers real backend work (retrieval, an LLM call) and the query text can be long/contain special characters, which fits a POST body far better than a URL query string, and POST is the conventional choice for an action with side effects (here, logging the query) rather than a pure, cacheable data fetch.

14.10 Networking

Q14. What is CORS, and how does it apply to your project specifically?

Answer: CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a page from one origin from calling an API on a different origin unless that API explicitly allows it.

Technical Explanation: Here, the frontend runs on localhost:5173 and the backend on localhost:8000 - different origins by browser rules - so app.py explicitly adds CORSMiddleware allowing those two exact origins.

14.11 Security

Q15. How are user passwords protected in your database?

Answer: Never stored in plain text - each password is hashed with PBKDF2-HMAC-SHA256 using 120,000 iterations and a unique random 16-byte salt per user; only the salt and hash are stored.

Possible follow-up: Why not just use SHA-256 once?

Follow-up answer: A single SHA-256 hash is extremely fast to compute, making brute-force/dictionary attacks on a stolen database feasible at high speed; PBKDF2's repeated iterations deliberately slow down each individual guess.

14.12 Testing

Q16. How did you test this project?

Answer: Primarily through manual testing via the frontend, and through CLI test blocks built into several service files (e.g. running hybrid_service.py directly opens an interactive query loop), plus the evaluation benchmark scripts, which validate retrieval QUALITY rather than code correctness.

What I should remember: Be honest that there is no automated pytest suite - this is a real, fair area for improvement, not something to claim you have.

14.13 Deployment

Q17. How would you deploy this to production?

Answer: Currently it only runs locally. To deploy, I would containerize the backend (Dockerfile), move from SQLite to a networked database such as PostgreSQL for multi-instance safety, host ChromaDB as a persistent service or managed vector database, update CORS origins to the real frontend domain, serve the built frontend as static files (or via a CDN) instead of the Vite dev server, and either self-host Ollama on a GPU instance or swap in a hosted LLM API depending on cost/latency requirements.

What I should remember: This is a design answer showing you understand the gap between 'runs on my machine' and production - don't claim any of this is already built.

14.14 Performance

Q18. Where are the main latency costs in a single query, and how do you know?

Answer: The pipeline measures and returns per-stage latency explicitly: retrieval_ms, reranking_ms, prompt_ms, and generation_ms.

Technical Explanation: Generation (the local LLM call) is typically the largest cost in a RAG pipeline; the BM25 cache and the small (50-candidate) retrieval/fusion/rerank stages are designed to be comparatively

fast.

Possible follow-up: How would you reduce generation latency specifically?

Follow-up answer: Options include using a smaller/faster LLM, quantizing the model further, reducing num_predict (the max generated tokens), or streaming the response to the user incrementally instead of waiting for the full generation to complete.

14.15 Scalability

Q19. Would this architecture scale to many concurrent users?

Answer: Not without changes. SQLite and the in-process BM25 index are well suited to a single server instance but become a bottleneck across multiple instances; the LLM call is also inherently sequential per-GPU unless you add request batching or multiple model replicas.

What I should remember: Name the specific bottlenecks (SQLite single-writer behaviour, one in-memory BM25 index per process, one local Ollama instance) rather than giving a generic answer.

14.16 ML / AI (present in this project)

Q20. Why ExtraTreesRegressor instead of a neural network for the adaptive weight predictor?

Answer: The problem is small tabular regression - 17 numeric features predicting one number - which tree ensembles handle very well; a neural network would add training complexity and data requirements without a clear benefit at this scale.

Technical Explanation: The training script even trains and compares a RandomForestRegressor alongside the ExtraTreesRegressor to justify the choice empirically rather than by default.

14.17 Other Technologies Actually Used

Q21. Why run the LLM locally with Ollama instead of calling a hosted API like OpenAI's?

Answer: It avoids per-token API costs and keeps data fully on-device, at the cost of a smaller, generally less capable model (Qwen2.5:7B) than large hosted models.

What I should remember: A fair, two-sided answer like this is stronger than only listing the benefits.

Top 30 Most Important Interview Questions

Organized in a logical preparation order: start broad, then go deep into retrieval, ML, data, and engineering practice.

Q1. In one or two sentences, what does your project do and what is its main contribution?

Answer: It's an adaptive hybrid RAG system: instead of a fixed BM25/dense weight, a trained ML model predicts the best fusion weight per query, improving retrieval quality over a fixed baseline.

Why an interviewer may ask this: This is always the opening question.

Key points to remember:

- State the problem (fixed weight is suboptimal)
- state the fix (learned per-query weight)
- mention it's evaluated honestly, not just claimed

Possible follow-up: How did you validate that it actually works better?

Follow-up answer: With a 1,000-query, 4-domain benchmark comparing Fixed Hybrid vs Adaptive Hybrid using Precision@10, Recall@10, MRR, and nDCG@10.

Q2. Why is a fixed BM25/Dense weight a real problem?

Answer: Different queries genuinely favour different retrieval signals - keyword-heavy queries favour BM25, paraphrased/conceptual queries favour dense retrieval - so no single fixed ratio is optimal for every query.

Why an interviewer may ask this: Tests whether you understand the motivation, not just the mechanism.

Key points to remember:

- Keyword vs semantic queries need different weighting
- a fixed ratio is a compromise

Possible follow-up: Can you give a concrete example of each query type?

Follow-up answer: A query like an exact product code or name is keyword-heavy (BM25 wins); a paraphrased conceptual question like 'how does the body regulate blood sugar' is semantic (dense retrieval wins).

Q3. Walk me through what happens, step by step, from a user typing a question to receiving an answer.

Answer: Query -> parallel BM25 (top-50) and dense retrieval (top-50) -> 17 features extracted -> ExtraTreesRegressor predicts the fusion weight -> scores normalized and fused -> top-10 kept -> cross-encoder reranks to top-5 -> a grounded prompt is built -> Qwen2.5:7B generates the answer.

Why an interviewer may ask this: The single most important 'explain your project' question - be ready to say this fluently.

Key points to remember:

- Name every stage in order
- know what CANDIDATE_K and TOP_K are (50 and 10)

Possible follow-up: What would happen if you skipped the reranking step entirely?

Follow-up answer: You would keep the fusion order as final; reranking generally improves ordering accuracy because a cross-encoder can directly compare the query and document jointly, so skipping it would likely reduce answer quality slightly, though the pipeline would still function.

Q4. What are the 17 features fed into the adaptive weight model, at a high level?

Answer: Five BM25 score statistics, five dense similarity statistics, two cross-retriever agreement features (overlap_ratio, rank_agreement), three query-shape statistics, and two lexical-coverage features.

Why an interviewer may ask this: Tests depth of understanding of the ML component's actual input.

Key points to remember:

- Know the 5 categories, don't need to recite all 17 names exactly
- know dense_top_lexical_coverage is the strongest reported feature

Possible follow-up: Which single feature turned out to matter most, according to your project's own analysis?

Follow-up answer: dense_top_lexical_coverage - the fraction of query words that literally appear in the top dense result - is reported in the README as the strongest individual feature in the V4 Rich model.

Q5. How were the training labels (the 'correct' fusion weight) generated for the adaptive model?

Answer: Not by hand-labeling. weight_optimizer.py sweeps many BM25/Dense weight combinations per query (0.0 to 1.0 in steps of 0.05), fuses with each, scores each with real IR metrics, and keeps whichever weight performed best as that query's 'oracle' label.

Why an interviewer may ask this: This is one of the more sophisticated, less-obvious parts of the project - expect this question.

Key points to remember:

- It's an oracle/brute-force search, not human labeling
- the found weight becomes the regression target

Possible follow-up: Isn't that circular - using retrieval metrics to generate labels, then training a model to predict a good weight?

Follow-up answer: It's not circular in the harmful sense: the oracle search is only feasible offline with ground-truth relevance judgments and unlimited compute per query; the trained model has to predict the SAME quality of weight instantly, from features only, without doing that expensive search at request time - so the model is learning to approximate an expensive offline process cheaply at inference time.

Q6. Why ExtraTreesRegressor specifically, and how is it different from Random Forest?

Answer: Both are tree ensembles; Extra Trees additionally randomizes the SPLIT THRESHOLD (not just the feature subset) at each node, which increases randomness, typically reduces variance further, and trains faster.

Why an interviewer may ask this: Tests real ML knowledge versus name-dropping.

Key points to remember:

- Random Forest: random features, best split
- Extra Trees: random features, random split threshold too

Possible follow-up: Did you compare it against Random Forest? What did you find?

Follow-up answer: Yes - train_v4_rich_model.py trains both with identical hyperparameters on the same train/validation split and compares MAE/RMSE/R² to justify the choice empirically.

Q7. What do BM25 and dense retrieval each individually miss, and why do you need both?

Answer: BM25 misses paraphrases/synonyms (it only matches literal words); dense retrieval can miss queries that need an exact keyword, code, or name match, because embeddings compress meaning and can lose precise lexical detail.

Why an interviewer may ask this: Foundational RAG question.

Key points to remember:

- BM25 = lexical/exact

- dense = semantic/conceptual
- combining covers more query types

Possible follow-up: Can hybrid retrieval ever be worse than either retriever alone?

Follow-up answer: Yes, in principle a poorly-weighted fusion could dilute a retriever that was already correct - this is exactly why the adaptive weighting exists, to avoid always applying a compromise weight.

Q8. How are the two independent retrieval scores (BM25 score and dense distance) made comparable before fusion?

Answer: Dense distance is converted to a similarity via $1/(1+\text{distance})$ (lower distance -> higher similarity), then BOTH BM25 scores and dense similarities are independently min-max normalized to $[0, 1]$ before being combined with the adaptive weights.

Why an interviewer may ask this: Tests whether you understand WHY normalization is necessary, not just that it happens.

Key points to remember:

- Raw scales are incompatible
- normalize each signal independently, then weight and sum

Possible follow-up: What happens if all scores in a batch are identical?

Follow-up answer: `min_max_normalize` returns an array of all 1.0s when max equals min, avoiding a divide-by-zero and treating a flat batch as maximally 'confident' rather than crashing.

Q9. How does your fusion step avoid duplicating a document that both BM25 and dense retrieval found?

Answer: It merges results into a Python dict keyed by a stable document identifier (`chunk_id`, then `document_id`, then retriever ID, then a text hash as a last resort), so a document found by both retrievers updates one merged row instead of creating two.

Why an interviewer may ask this: Tests understanding of a specific, real engineering detail.

Key points to remember:

- Hash map join by stable key
- avoids $O(n^2)$ comparison

Possible follow-up: What if the same chunk had a different ID from each retriever?

Follow-up answer: Then the merge could fail to recognize them as the same document and treat them as two separate candidates - this is exactly why the code prioritizes `chunk_id/document_id` metadata over the retriever's own raw ID.

Q10. Why is a cross-encoder used for reranking instead of for the initial retrieval?

Answer: A cross-encoder jointly processes (query, document) pairs, which is more accurate but far too slow to run over an entire corpus of tens of thousands of documents - so it's used only to re-score a small shortlist after retrieval has already narrowed the field.

Why an interviewer may ask this: Standard RAG architecture question.

Key points to remember:

- Bi-encoder = fast, used for initial retrieval
- cross-encoder = accurate but slow, used for reranking a shortlist

Possible follow-up: What's the latency trade-off of adding this stage?

Follow-up answer: It adds real latency (one forward pass per candidate pair), but only over a small top-K shortlist (5-10 items in this project), so the cost is bounded and generally worth the accuracy gain.

Q11. What does the grounded-QA system prompt actually instruct the model to do?

Answer: To answer only from the provided document context, never use outside knowledge, never invent facts/examples/numbers, explicitly state when the context doesn't fully answer the question, and never mention the retrieval system itself in the answer.

Why an interviewer may ask this: Tests whether you actually understand your own prompt engineering.

Key points to remember:

- Grounding rules
- explicit 'say when context is insufficient' instruction
- don't leak internal system details

Possible follow-up: How would you verify the model is actually following these instructions?

Follow-up answer: I don't currently have an automated check for this - it's enforced only through prompt design and a low temperature, which is a known limitation; a real improvement would be adding an automated groundedness/entailment check between the answer and the retrieved context.

Q12. Why is the generation temperature set to 0.1?

Answer: A low temperature makes the model's output more deterministic and less likely to deviate creatively from the retrieved context, which supports the grounded-answer requirement.

Why an interviewer may ask this: Tests LLM parameter understanding.

Key points to remember:

- Low temperature = more deterministic/focused, not more 'confident' in a factual sense

Possible follow-up: What would happen with temperature 1.0 instead?

Follow-up answer: The model would sample more randomly/creatively, increasing the chance of wording that drifts from the source text or introduces unsupported details - undesirable for a grounded-QA use case.

Q13. Why does the pipeline retrieve 50 candidates from each retriever but only keep the top 10 after fusion?

Answer: A wider candidate pool (CANDIDATE_K=50) gives the adaptive fusion step more documents to reorder from, since a document that ranked, say, 30th in dense retrieval but is actually highly relevant lexically could still surface into the final top 10 after fusion.

Why an interviewer may ask this: Tests understanding of why the two K values (CANDIDATE_K and TOP_K) are different.

Key points to remember:

- CANDIDATE_K=50 (recall-oriented)
- TOP_K=10 (precision-oriented final answer set)

Possible follow-up: What would happen if CANDIDATE_K were reduced to 10?

Follow-up answer: Fusion would have far less room to reorder - if a relevant document ranked, say, 15th in one retriever, it would never even be considered, likely reducing recall.

Q14. How does document upload and per-user retrieval work?

Answer: An uploaded PDF/DOCX/TXT is extracted to text (pypdf/python-docx), split into overlapping 500-character chunks, embedded, and stored in a Chroma collection scoped to that user; questions are then answered using the same BM25+dense+adaptive-fusion pattern, but scoped only to that user's documents, plus a context-expansion step.

Why an interviewer may ask this: Full ingestion pipeline understanding.

Key points to remember:

- Extraction -> chunking -> embedding -> per-user collection -> scoped retrieval -> context expansion

Possible follow-up: Why is context expansion needed specifically for user documents but effectively invisible for the shared corpus?

Follow-up answer: Because a user's own document is often read start-to-finish as connected prose, so an isolated top-ranked chunk can miss surrounding context the LLM needs; the shared corpus is built from many independent documents where this effect is less central to the design, though the same technique could in principle be applied there too.

Q15. Why 500-character chunks with 100-character overlap specifically?

Answer: It's a fixed, simple choice that balances having enough context per chunk against keeping chunks small and specific enough for precise retrieval; the overlap prevents a sentence or idea from being fully lost at a chunk boundary.

Why an interviewer may ask this: Tests whether you understand this was a deliberate, if simple, design choice.

Key points to remember:

- Sliding window
- `chunk_size=500, overlap=100`
- trade-off: bigger chunks = more context but less precise retrieval

Possible follow-up: What's a more sophisticated alternative?

Follow-up answer: Semantic or sentence/paragraph-aware chunking, which respects natural text boundaries instead of a fixed character count - a real potential improvement I'd consider next.

Q16. Describe your authentication system end to end.

Answer: SQLite-backed: registration hashes the password with PBKDF2-HMAC-SHA256 (120,000 iterations, unique salt per user) and stores only the hash+salt; login verifies with a timing-safe comparison and issues a random, opaque Bearer session token stored in a sessions table; protected routes require that token in the Authorization header.

Why an interviewer may ask this: Full-stack security understanding.

Key points to remember:

- PBKDF2, salt, `hmac.compare_digest`, opaque tokens (not JWT)

Possible follow-up: Why opaque tokens instead of JWTs?

Follow-up answer: Opaque tokens require a database lookup per request but can be instantly revoked (just delete the row); JWTs avoid the lookup but are harder to revoke before they naturally expire, since the token itself is self-contained and valid until its signature-checked expiry.

Q17. What happens in the database when a user resets their password?

Answer: The reset token is validated (exists, not expired, not already used) -> a new hash+salt is generated and saved -> the reset token is marked used -> critically, every existing session for that user is deleted from the sessions table, logging out all devices.

Why an interviewer may ask this: Tests specific SQL/security flow understanding.

Key points to remember:

- Token validated once (single-use)
- sessions wiped on reset - real security practice

Possible follow-up: Why is deleting all sessions important here?

Follow-up answer: If the password was compromised and is now being reset for that reason, any session token an attacker already obtained needs to stop working immediately, not remain valid until it separately expires.

Q18. What indexes exist in your database schema, and what do they speed up?

Answer: `idx_conversations_user` on `conversations(user_id)` and `idx_messages_conversation` on `messages(conversation_id)` - both created explicitly, speeding up 'list this user's conversations' and 'get all messages in this conversation' respectively.

Why an interviewer may ask this: Tests DBMS fundamentals applied to your actual schema.

Key points to remember:

- Two explicit indexes
- `users/sessions` tables rely on implicit indexes from `UNIQUE` constraints

Possible follow-up: What query pattern would NOT benefit from these indexes?

Follow-up answer: A query filtering or sorting by a column that isn't indexed - for example, searching messages by their text content would still require a full table scan, since there's no index on the `content` column.

Q19. Why does your project use two separate SQLite database files instead of one?

Answer: `app.db` holds authentication data (`users`, `sessions`, `tokens`) and `rag_app.db` holds conversation/chat data - they were built as two separate concerns by two different service modules, each initializing its own schema.

Why an interviewer may ask this: Tests whether you can reason about a real design choice, even an implicit one.

Key points to remember:

- Separation by concern, not a hard technical requirement

Possible follow-up: Would you combine them into one database file? Why or why not?

Follow-up answer: I could combine them into one file with more tables - SQLite handles that fine - but keeping them separate does make the two concerns (identity vs. conversation history) easier to reason about and migrate independently if needed.

Q20. How does ChromaDB actually find the most similar documents to a query?

Answer: It stores each document's embedding vector, and given a query embedding, it computes similarity/distance to the stored vectors and returns the closest ones - a nearest-neighbour search over the vector space.

Why an interviewer may ask this: Basic vector-DB literacy check.

Key points to remember:

- Vectors in, distances/similarities out
- conceptually nearest-neighbour search

Possible follow-up: How is this fundamentally different from a SQL `WHERE` clause?

Follow-up answer: A SQL `WHERE` clause matches exact or range conditions on structured columns; vector similarity search finds geometrically 'close' vectors, which corresponds to semantic closeness in meaning rather than any exact field match.

Q21. What is your evaluation methodology, and what did it show?

Answer: A 1,000-query, 4-domain (Argumentation, Finance, Medical, Scientific) benchmark comparing BM25-only, Dense-only, Fixed Hybrid, and Adaptive Hybrid, using `Precision@10`, `Recall@10`, `MRR`, and `nDCG@10` that I implemented myself. Adaptive Hybrid improved `Precision@10` (+3.33%), `Recall@10` (+3.70%), and `nDCG@10` (+2.09%), but `MRR` was slightly lower (-0.40%).

Why an interviewer may ask this: The single most important evaluation question - know these numbers.

Key points to remember:

- 4 configurations compared

- 4 standard IR metrics
- honest about the MRR regression

Possible follow-up: Why might MRR specifically be the metric that didn't improve?

Follow-up answer: MRR only cares about the rank of the FIRST relevant result, while Precision/Recall/nDCG@10 consider the whole top-10 list - it's possible the adaptive method improves the overall quality of the top-10 set without always keeping the single best result in the very top rank, which is a plausible, honest explanation rather than a definitive proven cause.

Q22. Is the adaptive approach better for every query? How do you know?

Answer: No, and the project's own per-query analysis shows this directly: out of 1,000 queries, MRR improved for 138, stayed equal for 768, and actually decreased for 94.

Why an interviewer may ask this: Tests whether you'll overclaim or answer honestly with evidence.

Key points to remember:

- Cite the actual per-query breakdown, not just the averages

Possible follow-up: How would you investigate the 94 queries that got worse?

Follow-up answer: I would look at their feature values (e.g. was overlap_ratio unusually high, meaning the two retrievers already agreed and fusion mattered less) and check whether they share a query-length or domain pattern, which the project has evaluation scripts specifically aimed at (error_analysis_v3.py, group_v3_failures.py).

Q23. What are the four evaluated domains and what did the domain-level results show?

Answer: Argumentation, Finance, Medical, and Scientific. The README's domain table shows retrieval quality varies substantially by domain - for example, Scientific has much higher MRR (0.6170) than Medical (0.4303) - which is itself evidence for why query-dependent weighting is worth pursuing.

Why an interviewer may ask this: Tests whether you read your own results carefully.

Key points to remember:

- 4 domains
- big variance between domains supports the adaptive-weighting motivation

Possible follow-up: Why might Medical queries score lower than Scientific ones on MRR?

Follow-up answer: This isn't something the uploaded files establish conclusively; a reasonable hypothesis is that domain vocabulary overlap between queries and documents, or the difficulty/ambiguity of medical terminology, differs between domains - but I would say this is a hypothesis, not a proven finding from the project as uploaded.

Q24. What is precision@10 and recall@10, in plain terms?

Answer: Precision@10 asks: of the top 10 results you returned, what fraction were actually relevant? Recall@10 asks: of all the relevant documents that exist for this query, what fraction did you manage to include in your top 10?

Why an interviewer may ask this: Basic IR metric literacy - must be fluent here.

Key points to remember:

- Precision = correctness of what you returned
- Recall = completeness of what you returned

Possible follow-up: Why can precision and recall trade off against each other?

Follow-up answer: Returning more results generally increases the chance of catching more relevant ones (helping recall) but also increases the chance of including irrelevant ones (hurting precision) - though at a FIXED cutoff like @10, this project's numbers show both moved in the same direction (both improved) for the adaptive method.

Q25. What is nDCG and why does it matter more than plain precision for a ranking system?

Answer: nDCG (normalized Discounted Cumulative Gain) rewards relevant results MORE when they appear near the top of the ranked list, not just when they appear somewhere in the top-K at all - it captures ranking QUALITY, not just set membership.

Why an interviewer may ask this: Tests deeper IR metric understanding beyond precision/recall.

Key points to remember:

- DCG discounts by rank position ($\log_2$)
- nDCG normalizes against the best possible ranking (IDCG)

Possible follow-up: How is nDCG computed in your code?

Follow-up answer: evaluation/metrics.py implements `dcg_at_k()` by summing $1/\log_2(\text{rank}+1)$ for each relevant document found, then `ndcg_at_k()` divides that by the DCG of the ideal (best-possible) ordering of the same relevant documents.

Q26. What is the biggest limitation of this project as it currently stands?

Answer: The trained model, the vector index, and the BM25 cache are not included in the repository, so the project cannot be run end-to-end from a fresh clone without first rebuilding those artifacts - and there is no automated test suite.

Why an interviewer may ask this: Tests self-awareness and honesty under direct questioning.

Key points to remember:

- Reproducibility gap
- no automated tests
- MRR regression under adaptive fusion

Possible follow-up: If you had one more week, what would you fix first?

Follow-up answer: I would write a single setup script that builds the corpus index, trains the adaptive model, and verifies everything end-to-end from a fresh clone, since that removes the biggest barrier to anyone else actually running the project.

Q27. Why did you build a full authentication and conversation-history system around a retrieval research idea?

Answer: To make the adaptive retrieval idea usable as a real product experience rather than only a benchmark script - a user can register, upload their own documents, and have a persistent chat history, which also let me practice full-stack engineering (auth security, relational schema design, a React frontend) around the research core.

Why an interviewer may ask this: Tests whether you can explain scope decisions, not just individual features.

Key points to remember:

- Research core + product shell
- shows both ML/IR skill and full-stack skill

Possible follow-up: If this were purely a research paper, what would you have left out?

Follow-up answer: Almost everything except the retrieval/fusion/evaluation code - authentication, conversation persistence, and the dashboard UI exist because I wanted a demonstrable, usable application, not just a benchmark script.

Q28. What was the hardest technical problem you had to solve or understand in this project?

Answer: Personalize this truthfully based on your own experience building or studying the code; a strong, defensible example grounded in the code is the fusion/normalization logic - making two incompatible score

scales (BM25 relevance scores and Chroma vector distances) comparable and combinable safely, including edge cases like a batch where every score is identical.

Why an interviewer may ask this: Very common interview question - must have a real, specific, project-grounded answer ready.

Key points to remember:

- Pick ONE concrete example you can explain in depth, not a vague generality

Possible follow-up: How did you verify your solution was actually correct?

Follow-up answer: By reasoning through edge cases in the normalization code (e.g. what happens when max equals min) and by checking the resulting fusion scores and rankings against expectations during manual CLI testing (hybrid_service.py's own `__main__` block).

Q29. What would you build next if you continued this project?

Answer: A reproducible one-command setup script, an automated test suite, deeper error analysis on the queries where adaptive fusion underperforms, and possibly sentence-aware chunking to replace the fixed-size chunker.

Why an interviewer may ask this: Forward-looking question - shows genuine understanding versus rote memorization of what already exists.

Key points to remember:

- Reproducibility
- testing
- failure analysis
- better chunking

Possible follow-up: Which of those would you prioritize first and why?

Follow-up answer: The reproducible setup script, because without it the project's strong evaluation results aren't verifiable by anyone else, which undermines the research credibility of everything else.

Cross-Questions ("Why? How? What if?")

These simulate an interviewer who keeps digging. Each chain starts from a natural opening question and pushes several levels deeper, with grounded answers.

Interviewer: Why did you use BM25 instead of just TF-IDF?

Follow-up 1: BM25 improves on plain TF-IDF by saturating term-frequency reward and normalizing for document length, which generally gives better ranking quality - and rank-bm25's BM25Okapi is a standard, well-tested implementation, so there was no need to hand-roll TF-IDF ranking.

Follow-up 2: What if two documents have the exact same BM25 score?

Follow-up 3: Python's `np.argsort` is stable for equal values in a well-defined but implementation-specific way; the project does not add an explicit tie-breaking rule beyond that, which is a fair thing to acknowledge as an edge case rather than claim is deliberately handled.

Interviewer: Why not just always use dense retrieval and skip BM25 entirely?

Follow-up 1: Dense retrieval can miss queries needing exact keyword/code/name matches that don't embed distinctively; the whole point of the project is that combining both, weighted per query, beats either alone or a fixed blend.

Follow-up 2: What if the corpus is very small - would hybrid retrieval still help?

Follow-up 3: For a very small corpus, both retrievers would likely already find the small set of relevant documents easily, so the marginal benefit of adaptive weighting would likely shrink - this project's benchmark corpus is large (75,000+ chunks), where retrieval difficulty is more meaningful.

Interviewer: Why did you choose 500-character chunks specifically?

Follow-up 1: It's a fixed, simple choice balancing enough context per chunk against retrieval precision, with 100-character overlap to avoid losing content at chunk boundaries.

Follow-up 2: What happens as the corpus grows to millions of documents?

Follow-up 3: More chunks means a larger BM25 index (more memory, slower `get_scores()` per query since it scores the whole corpus) and a larger ChromaDB collection (index build and query time both grow) - at that scale you would likely need approximate nearest-neighbour tuning and possibly a distributed BM25 approach rather than the current single in-memory index.

Follow-up 4: How would you optimize retrieval at that scale?

Follow-up 5: Options include sharding the BM25 index, using ChromaDB's ANN index parameters more aggressively, reducing CANDIDATE_K if latency became a problem, or moving to a purpose-built, horizontally scalable search engine.

Interviewer: Why did you use SQLite instead of PostgreSQL?

Follow-up 1: SQLite needs no separate server process, is simple to set up for a single-instance local project, and fully supports the SQL features (foreign keys, transactions) this project actually needs.

Follow-up 2: What happens when the data grows or many users write concurrently?

Follow-up 3: SQLite allows only one writer at a time on the whole database file; under heavy concurrent write load (many simultaneous logins, many simultaneous conversations being updated) this becomes a real bottleneck.

Follow-up 4: How would you optimize it?

Follow-up 5: Migrate to PostgreSQL (or MySQL), which supports proper concurrent multi-writer access, connection pooling, and would let the backend run as multiple horizontally-scaled instances behind a load balancer.

Interviewer: Why PBKDF2 instead of bcrypt or Argon2 for password hashing?

Follow-up 1: PBKDF2-HMAC-SHA256 is available directly from Python's standard library (`hashlib.pbkdf2_hmac`) with no extra dependency, and with a high iteration count (120,000) it provides solid protection.

Follow-up 2: Why not another approach?

Follow-up 3: `bcrypt` and Argon2 are generally considered stronger modern choices (Argon2 in particular is memory-hard, making GPU/ASIC cracking harder) - a fair, honest answer is that PBKDF2 was likely chosen for its zero-dependency convenience, and upgrading to Argon2 (via a library like `argon2-cffi`) would be a reasonable future improvement.

Interviewer: Why does the adaptive model predict only a single BM25 weight and derive the dense weight as 1 minus it, instead of predicting both independently?

Follow-up 1: Constraining `dense_weight = 1 - bm25_weight` guarantees the two weights always sum to 1 by construction, which keeps the fusion score properly bounded and interpretable as a weighted average, and it also halves the model's prediction problem to one target instead of two.

Follow-up 2: What if you wanted the weights to NOT sum to 1 - would that ever make sense?

Follow-up 3: Allowing weights to sum to something other than 1 (e.g. boosting both signals when both retrievers are highly confident, or dampening both when neither is) is a plausible extension, but it changes the fusion formula's interpretation and wasn't the design chosen here - the current design intentionally keeps fusion as a simple weighted average.

Interviewer: Why min-max normalization instead of z-score (standard score) normalization for fusion?

Follow-up 1: Min-max normalization maps scores into a fixed `[0, 1]` range, which is convenient because both signals then live on the exact same bounded scale before being weighted and summed.

Follow-up 2: What's a downside of min-max normalization specifically?

Follow-up 3: It's sensitive to outliers - a single unusually high or low score in the batch stretches or compresses everything else's normalized value, whereas z-score normalization is comparatively more robust to a single extreme value (though still not fully robust, since mean and standard deviation are also outlier-sensitive).

Interviewer: Why does the project use a cross-encoder for reranking rather than simply trusting the fusion score ordering?

Follow-up 1: A cross-encoder can jointly attend over the query and document text together, which is generally more accurate at judging fine-grained relevance than combining two independently-computed retrieval scores.

Follow-up 2: What if the cross-encoder disagrees strongly with the fusion ranking - which one should you trust more?

Follow-up 3: The project trusts the reranker's ordering for the final answer (the reranked list feeds the prompt), on the reasoning that a purpose-built relevance model looking at both texts together should be more accurate than an earlier-stage heuristic combination - though this hasn't been separately ablation-tested in the evaluation as uploaded.

Interviewer: Why generate the answer with a local 7B model instead of a larger hosted model?

Follow-up 1: Running locally through Ollama avoids per-token API costs and keeps generation fully on-device.

Follow-up 2: What if answer quality on complex questions isn't good enough?

Follow-up 3: You would need a larger/more capable model, which means either more local GPU compute or switching to a hosted API - a real cost/quality/privacy trade-off that would need to be decided based on the actual deployment requirements.

Interviewer: Why does the BM25 index get cached to a pickle file on disk instead of rebuilt every time?

Follow-up 1: Rebuilding BM25Okapi over 75,000+ documents means re-tokenizing and re-indexing the entire corpus, which is slow to do on every server restart; a validated on-disk cache skips that when the underlying data hasn't changed.

Follow-up 2: What if the corpus changes but the cache isn't invalidated correctly?

Follow-up 3: The cache validates a fingerprint (a hash of the collection name, document count, and 3 sampled document IDs) before trusting it - if the corpus changed in a way that alters that fingerprint, the cache is correctly rejected and rebuilt; if it changed in a way the lightweight fingerprint can't detect (e.g. only middle documents changed without changing the 3 sampled positions), a stale cache could theoretically be used, which is a known trade-off of this lightweight fingerprinting approach versus hashing the full corpus.

Interviewer: Why generate the training labels for the adaptive model via a grid search instead of using human relevance judgments directly as the model's target?

Follow-up 1: The oracle grid search directly answers the exact question the model needs to learn - 'what fusion weight produces the best retrieval quality for this query' - which is a more direct training signal than asking humans to guess a numeric weight, something humans have no intuitive sense for.

Follow-up 2: What if the step size of 0.05 in the grid search misses the true optimal weight?

Follow-up 3: It would introduce some label noise/quantization error - the true optimum for a given query might fall between two tested weights - but with 21 tested values (0.00 to 1.00 in steps of 0.05) this is a reasonably fine-grained search, and any single-query imprecision would tend to average out across the large training set used to fit the regressor.

"What Did You Personally Do?" Preparation

IMPORTANT: The uploaded ZIP is a snapshot of the finished code. It contains no commit history, no contributor list, and no comments attributing specific files to specific people. This means I CANNOT determine from the files alone which parts you personally wrote versus anyone else who may have been involved. Every question in this section must be answered based on YOUR actual memory of what you built, not by copying the wording below verbatim as if it were established fact.

17.1 Questions to prepare honestly

- What was your specific contribution to this project?
- Which files or modules did you personally implement?
- Which feature did you build end-to-end yourself?
- What specific technical decision did you make, and why?
- What problem did you personally debug or solve?
- What did you learn from building this?
- What would you change if you rebuilt it today?

17.2 How to prepare a truthful answer

For each question above, use this structure so your answer is both honest and specific: **(1) Name the exact component** (e.g. 'the adaptive fusion scoring logic in fusion_service.py'), **(2) Describe what it does** in your own words (you now have detailed technical descriptions throughout this document to draw on), **(3) Describe a specific decision or trade-off** you made while building it (e.g. why you chose min-max normalization, why you picked 500-character chunks), and **(4) Describe what you'd improve**, drawing on the honest 'potential improvements' list in Part 13.2 and Part 19.

17.3 A safe, honest template for 'what was your contribution?'

"I built [the specific part you actually built - be as specific as you honestly can, e.g. the retrieval and fusion pipeline / the authentication and database layer / the frontend / the evaluation benchmark]. The part I understand most deeply and can walk through in detail is [pick ONE real strength] - I can explain exactly how it works, why it was designed that way, and what I would improve about it."

HONESTY NOTE: Do not claim personal authorship of the entire project, every file, or every design decision unless that is genuinely true. If this was a solo project, say so plainly and confidently - you do not need to manufacture a division of labour that didn't exist. If it involved others, describe honestly which parts were yours.

17.4 What I learned (a template to personalize)

- How hybrid retrieval systems combine lexical and semantic search, and why a fixed fusion weight is a real limitation.
- How to engineer numeric features from unstructured retrieval output for a downstream ML model.
- How to generate training labels for a problem with no natural human-labeled ground truth (the oracle grid-search approach).
- How to design a secure authentication system by hand (password hashing, salting, session tokens) rather than treating auth as a black box.

- How to build and reason about a full-stack application connecting a React frontend, a FastAPI backend, a relational database, a vector database, and a locally-served LLM.
- How to evaluate a system honestly using standard information-retrieval metrics, including reporting a metric that did NOT improve.

Resume-Based Interview Questions

If this project appears on your resume, expect questions like the ones below. Each includes a grounded answer direction based on the actual code.

Q30. Explain this project.

Answer: Use the 1-minute pitch from Part 1.16 as your default answer length, and expand into the 2-3 minute version from Part 1.17 if the interviewer wants more detail.

Q31. Why did you build it?

Answer: A genuine, honest answer here should reflect your real motivation - for example, interest in retrieval-augmented generation and wanting to go beyond a basic RAG tutorial by addressing a real, documented limitation (fixed fusion weights) with an actual ML solution, evaluated properly rather than just assumed to work.

Q32. What was your role?

Answer: Answer based on Part 17's guidance - be specific and honest about which parts you personally built.

Q33. What technologies did you use, and why these specifically?

Answer: FastAPI for the backend API, SQLite for structured data, ChromaDB for vector search, rank-bm25 for lexical search, Sentence-Transformers for embeddings and reranking, scikit-learn for the adaptive weight model, Ollama+Qwen2.5:7B for local LLM generation, and React+Vite for the frontend - each chosen because it's a standard, well-supported tool for its specific job without unnecessary extra complexity (see Part 4 for the full WHY on each).

Q34. What was the hardest part?

Answer: Personalize this, but a strong, code-grounded candidate answer is the fusion/normalization logic - making two very differently-scaled retrieval signals (BM25 scores and vector distances) comparable and correctly merge-able without losing information about which retriever found what, plus designing the oracle grid-search approach to generate meaningful training labels for the adaptive weight model when no human-labeled weights existed.

Q35. What result did you achieve, and how did you measure it?

Answer: On a 1,000-query, 4-domain benchmark, Adaptive Hybrid retrieval improved Precision@10 by 3.33%, Recall@10 by 3.70%, and nDCG@10 by 2.09% over a Fixed Hybrid baseline, measured with Precision@10, Recall@10, MRR, and nDCG@10 implemented from scratch - though MRR was very slightly lower (-0.40%), which I report honestly rather than omit.

Q36. What did you learn?

Answer: Use Part 17.4's list and personalize it to what genuinely stood out to you.

Q37. What would you improve?

Answer: Reference Part 13.2's honest improvement list: reproducibility (missing build artifacts), automated testing, deeper failure analysis on the queries where MRR regressed, and more sophisticated chunking.

Q38. Can you explain this specific line/feature from your resume if I point to it?

Answer: Be ready to open any specific file mentioned in your resume bullet points and explain it live - this document's Part 3 (file-by-file) and Part 6 (input-processing-output flows) are written so you can reconstruct that explanation for any of the project's core files without needing to re-read the code live in the interview.

Q39. Can you demonstrate the project live?

Answer: Be upfront if you cannot, given the missing model/index artifacts noted in Part 12 - offer instead to walk through the code and architecture, or through the CLI test blocks (e.g. ``python -m services.hybrid_service``) if you have them set up and running locally.

Honesty Check - Claims You Should NOT Make

This section exists specifically to stop you from accidentally overclaiming in an interview. Everything below was found directly in the uploaded files.

19.1 Claims that ARE well-supported by the code (safe to make confidently)

- The adaptive weight predictor is a real, trained ExtraTreesRegressor using 17 engineered features - confirmed directly in `adaptive_predictor_v4.py` and `train_v4_rich_model.py`.
- The 1,000-query, 4-domain benchmark and its reported metrics come directly from the project's own README and are internally consistent with the metrics implementation in `evaluation/metrics.py`.
- Password hashing, salting, and session-invalidation-on-reset are real, correctly-implemented security practices, confirmed in `auth_service.py`.
- The context-expansion feature for user documents is real and specifically implemented in `services/user_rag/user_pipeline.py`.

19.2 Features mentioned in the README but that need a qualifier

"Email verification backend" / "Password reset backend"	These ARE implemented, but only as backend logic that returns the token directly in the API response for development purposes. There is no actual email-sending code (no SMTP client, no email service integration) anywhere in the ZIP. Say: "the backend logic and database schema for email verification exist, but actual email delivery was not implemented - the token is returned directly in development mode."
"Research dashboard"	This exists in the frontend (ResearchDashboard component) and is fed by real backend analytics endpoints (<code>/analytics/summary</code> , <code>/analytics/weights</code> , etc.) reading the JSONL retrieval logs - this claim is accurate.

19.3 Technologies listed but barely/not actually used (be precise)

Root requirements.txt contents (langchain, langgraph, langsmith, kubernetes, googleapis-common-protos, etc.)	These packages appear in the ROOT requirements.txt (which is oddly UTF-16 encoded) but a search of the actual backend code shows NO imports of langchain, langgraph, or langsmith anywhere. backend/requirements.txt is a much shorter, focused list that matches what's actually imported. Do NOT claim you used LangChain or LangGraph in this project - the retrieval/fusion/prompting pipeline is hand-built, not built on top of LangChain.
PyMuPDF	Listed in backend/requirements.txt and used only inside the LEGACY/prototype pdf_service.py, which is not imported by the live API pipeline (see 19.4 below). The actual live PDF extraction path uses pypdf, not PyMuPDF.

19.4 Legacy / unused prototype code (do not present as part of the live pipeline)

HONESTY NOTE: A direct search of the codebase confirms that `services/embedding_service.py`, `services/chunk_service.py`, `services/pdf_service.py`, and `services/vector_store.py` are NOT imported by any other file in the live API path (`app.py`, `api/`, or the current `hybrid_service.py / user_rag` pipeline). They read as earlier, single-PDF prototype scripts (they reference a hardcoded `'../data/raw/sample.pdf'` in their own test blocks). `services/document_loader.py` IS used, but only by offline training/statistics scripts (`training/generate_queries.py`, `services/corpus_statistics.py`), not by the live query-answering API. If asked to describe the live pipeline, describe `services/ingestion/` (`document_extractor.py`, `chunker.py`) and `services/user_rag/` - not these legacy files.

19.5 main.py vs app.py - do not conflate them

HONESTY NOTE: `backend/main.py` is a second, much smaller FastAPI app (6 lines, one route) that is separate from and not used by `app.py`. It appears to be an early scaffold file that was superseded by `app.py`. If asked "what is the entry point of your backend", the correct, well-supported answer is `app.py` (served via Uvicorn), not `main.py`.

19.6 Reproducibility / runnability

HONESTY NOTE: The uploaded ZIP does not contain the trained `adaptive_weight_model_v4_rich.pkl` file, the `chroma_db/` folder, or the `bm25_cache/` folder - these are runtime/build artifacts intentionally excluded from Git (per the `.gitignore` and `README`). This means the project as uploaded CANNOT answer a live query without first running corpus-building, training, and indexing steps. Do not claim you can demo it live from this ZIP alone without first setting those up.

19.7 Testing

HONESTY NOTE: No automated test suite (pytest or equivalent, with real assertions run in CI) was found anywhere in the ZIP. Files with 'test' in the name (e.g. `services/test_vector_db.py`) read as manual/ad-hoc verification scripts, not an automated regression suite. Do not claim the project has automated test coverage.

19.8 Deployment

HONESTY NOTE: No Docker configuration, CI/CD pipeline, or cloud deployment configuration was found. The project is documented and built for local development only. Do not claim it is deployed or production-ready without qualifying that this would require real additional engineering (see Part 14.13's deployment answer).

19.9 Metrics without further evidence

HONESTY NOTE: The `README`'s benchmark numbers (Precision@10, Recall@10, MRR, nDCG@10, and the domain-level and per-query improvement breakdowns) are presented as the project's own findings and this document treats them as such - but I (Claude) was not able to independently re-run the 1,000-query benchmark from the uploaded files, since the underlying trained model, vector index, and benchmark dataset artifacts were not included in the ZIP. Treat these as the project's self-reported research results, which is exactly how the `README` itself frames them - not as independently externally verified numbers.

19.10 LICENSE file

HONESTY NOTE: A `LICENSE` file exists in the repository but is empty (0 readable content found). The `README` states "This repository is currently intended as an academic/research project... Add an explicit license here if the project is later released for external reuse." Do not claim the project is released under a specific open-source license (MIT, Apache, etc.) - it currently is not.

Final Revision Sheet

PROJECT QUICK REVISION

PROJECT	Adaptive Hybrid RAG - a Retrieval-Augmented Generation system with a learned, per-query BM25/Dense fusion weight.
PROBLEM	Fixed hybrid-retrieval weights (e.g. always 50/50) are suboptimal because different queries genuinely favour different retrieval signals.
OBJECTIVE	Predict the right BM25/Dense fusion weight per query using a trained ML model, and prove it helps using proper IR evaluation.
SOLUTION	BM25 (rank-bm25) + Dense (ChromaDB + all-MiniLM-L6-v2) retrieval, 17 hand-engineered features, an ExtraTreesRegressor predicting the BM25 weight, min-max-normalized weighted fusion, cross-encoder reranking, and grounded generation with Qwen2.5:7B via Ollama.
TECH STACK	Python, FastAPI, SQLite, ChromaDB, rank-bm25, Sentence-Transformers, scikit-learn, Ollama, React 19, Vite.
ARCHITECTURE	Monolithic FastAPI backend, two SQLite databases, ChromaDB vector store, a persisted BM25 index, a locally-served LLM, and a React SPA frontend.
MAIN FEATURES	Adaptive hybrid retrieval, reranking, user document upload with per-user RAG, authentication, persistent conversations, latency measurement, a research/analytics dashboard, and a 1,000-query multi-domain benchmark.
DATABASE	SQLite (app.db: users/sessions/tokens; rag_app.db: conversations/messages) + ChromaDB (vector embeddings).
IMPORTANT ALGORITHMS	BM25 (Okapi), ExtraTreesRegressor, min-max normalization, PBKDF2 password hashing, and hand-implemented Precision/Recall/MRR/nDCG.
IMPORTANT APIs	POST /ask, POST /user/ask, POST /auth/login, POST /user/upload, POST /conversations/{id}/messages.
KEY CODE	fusion_service.py (score fusion), adaptive_predictor_v4.py (ML prediction), retrieval_feature_service.py (feature engineering), auth_service.py (secure auth).
MY CONTRIBUTION	[Fill in truthfully - see Part 17].
MAIN CHALLENGE	Making two differently-scaled retrieval signals comparable and mergeable, and generating meaningful ML training labels with no pre-existing human-labeled weights.
SOLUTION TO CHALLENGE	Independent min-max normalization + a stable-ID hash-map join for fusion; an oracle grid-search over weight combinations, scored by real retrieval metrics, for training labels.

RESULT	+3.33% Precision@10, +3.70% Recall@10, +2.09% nDCG@10 versus Fixed Hybrid, on a 1,000-query, 4-domain benchmark; MRR was -0.40% (reported honestly, not hidden).
LIMITATIONS	MRR regression, no automated tests, missing build artifacts in the repo (not reproducible from the ZIP alone), fixed-size (not semantic) chunking, no automated hallucination check.
FUTURE IMPROVEMENTS	One-command reproducible setup, automated test suite, deeper failure analysis on regressed queries, semantic/sentence-aware chunking, split the large App.jsx into components.

10 THINGS I MUST REMEMBER

- 1. The core idea: predict a per-query BM25/Dense fusion weight with ExtraTreesRegressor instead of using a fixed 50/50 blend.
- 2. CANDIDATE_K = 50 (candidates per retriever), TOP_K = 10 (final results kept after fusion).
- 3. Training labels for the adaptive model come from an ORACLE GRID SEARCH over weight combinations, not human labeling.
- 4. 17 features feed the model: BM25 stats, dense stats, overlap_ratio, rank_agreement, query stats, and lexical coverage (dense_top_lexical_coverage was reported as the strongest).
- 5. Fusion = independent min-max normalization of both signals, then a weighted sum, merged by a stable document key.
- 6. Benchmark results: Adaptive Hybrid improved Precision@10, Recall@10, and nDCG@10, but MRR was slightly LOWER - always report this honestly.
- 7. Authentication uses PBKDF2-HMAC-SHA256 (120,000 iterations) with per-user salts and opaque Bearer session tokens, not JWTs.
- 8. The LLM is Qwen2.5:7B, served fully locally through Ollama - no external API key is used anywhere.
- 9. Model artifacts, the vector index, and the BM25 cache are NOT included in the ZIP - the project isn't runnable end-to-end without rebuilding them first.
- 10. Some README-listed technologies (LangChain, LangGraph, PyMuPDF as the live PDF parser) are not actually used in the live code path - don't claim them.

5-MINUTE INTERVIEW REVISION

Read this immediately before walking in.

- **What it is:** A RAG system that learns a per-query BM25/Dense retrieval fusion weight instead of using a fixed ratio.
- **Pipeline:** Query -> BM25 top-50 + Dense top-50 (parallel) -> 17 features -> ExtraTreesRegressor predicts BM25 weight -> normalize + fuse -> top-10 -> cross-encoder reranks to top-5 -> grounded prompt -> Qwen2.5:7B (local, via Ollama) -> answer.
- **Training labels:** generated by an oracle grid search - sweep BM25/Dense weight combos per query, keep whichever performed best.
- **Result:** +3.33% Precision@10, +3.70% Recall@10, +2.09% nDCG@10 vs. Fixed Hybrid; MRR -0.40% (reported honestly).
- **Auth:** SQLite, PBKDF2 password hashing with per-user salts, opaque Bearer session tokens.

- **Extra feature:** user document upload with per-user Chroma collections and neighbouring-chunk context expansion.
- **Known gaps:** no automated tests; build artifacts (model/.pkl, chroma_db, bm25_cache) not in the repo; some listed dependencies (LangChain, LangGraph) aren't actually used.
- **If stuck on any question:** answer with what you know for certain, and if something genuinely isn't verifiable from the project, say so plainly rather than guessing - this document itself models that habit throughout.

This document was generated by analyzing the actual contents of adaptive-hybrid-rag-main.zip - its source code, README, requirements files, and configuration. Every technical claim is grounded in what was found in those files; every place where something could not be verified is marked as a Honesty Note rather than guessed. Good luck with your interviews.